

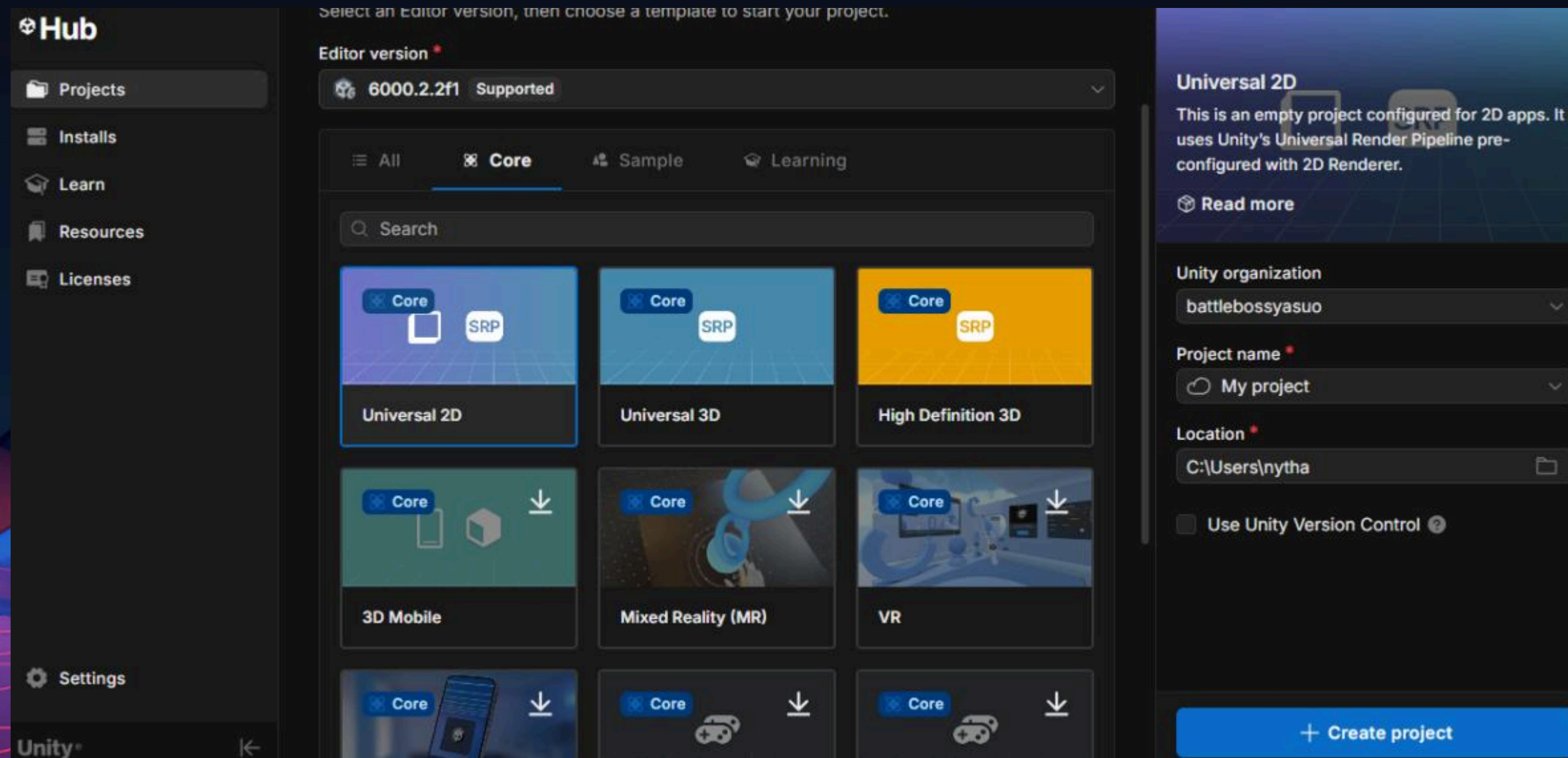


GNOME WELL STEP-BY-STEP

By: Abogado, Nythan
Abara, Jezreel
Cezar, Davidson

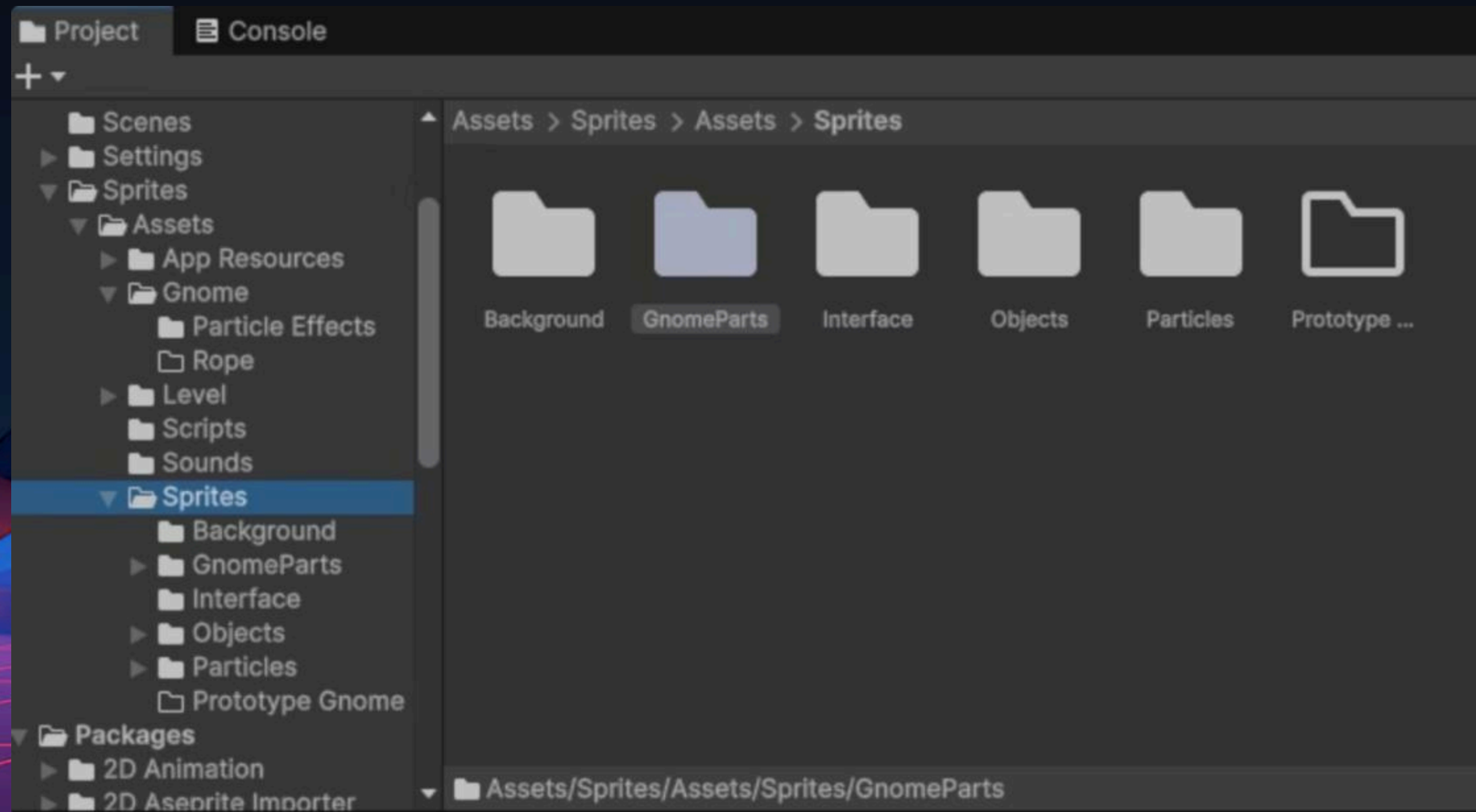
STEP 1

First create a new project in unity



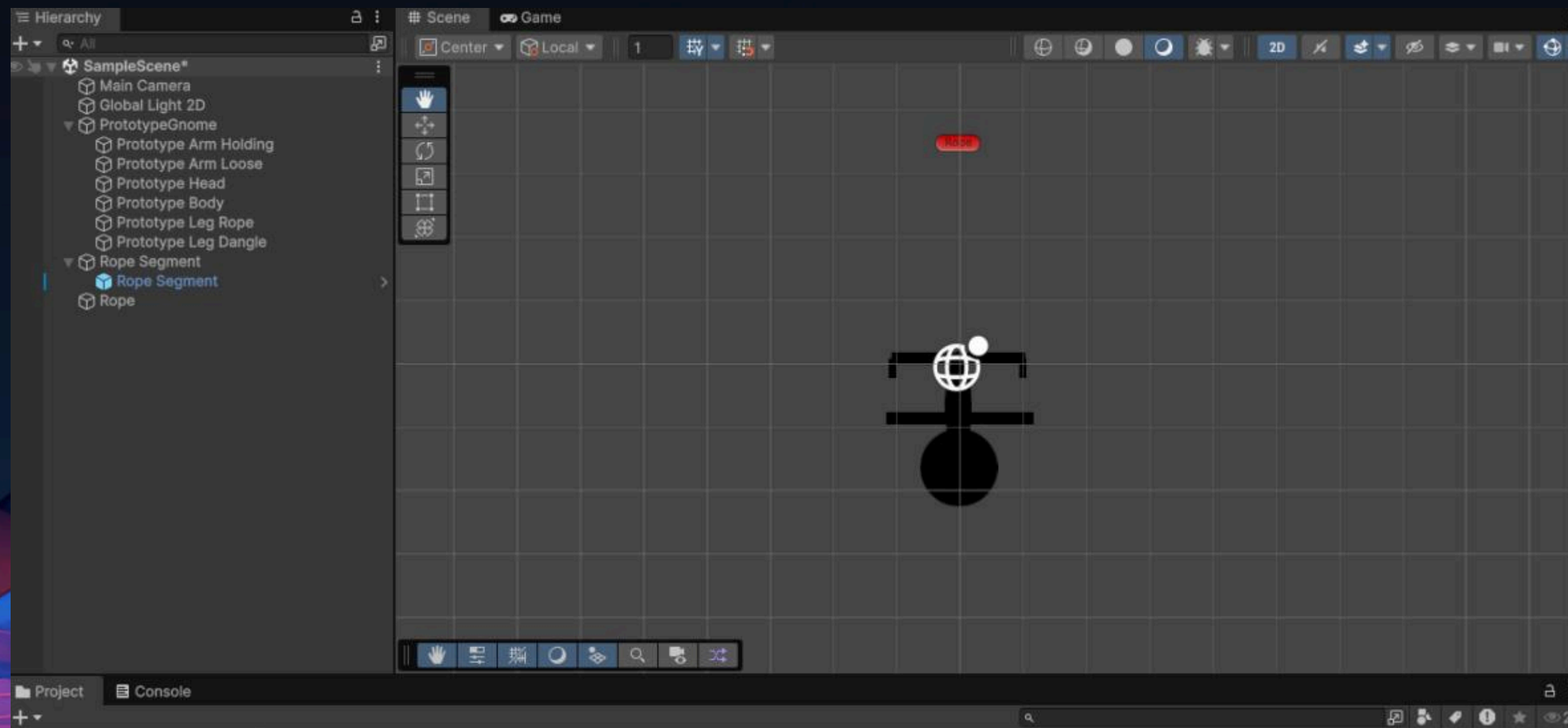
STEP 2

Import the files you
downloaded in the
book



STEP 3

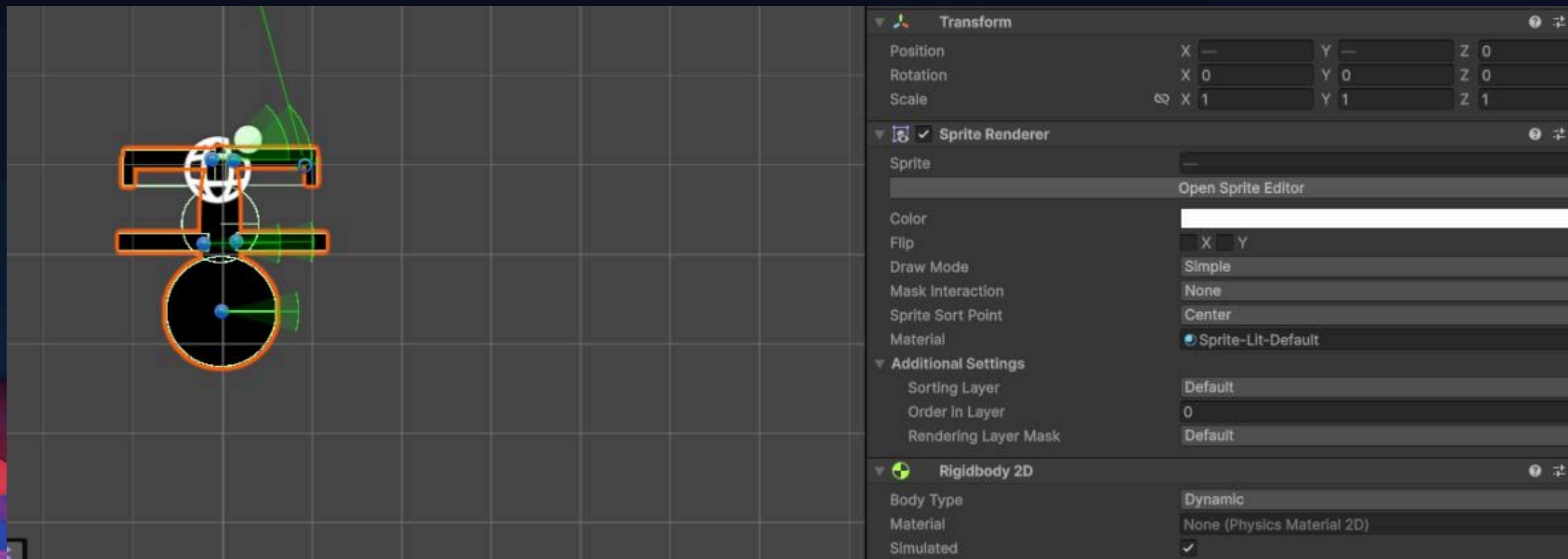
Add the assets, and make it like this



STEP 4

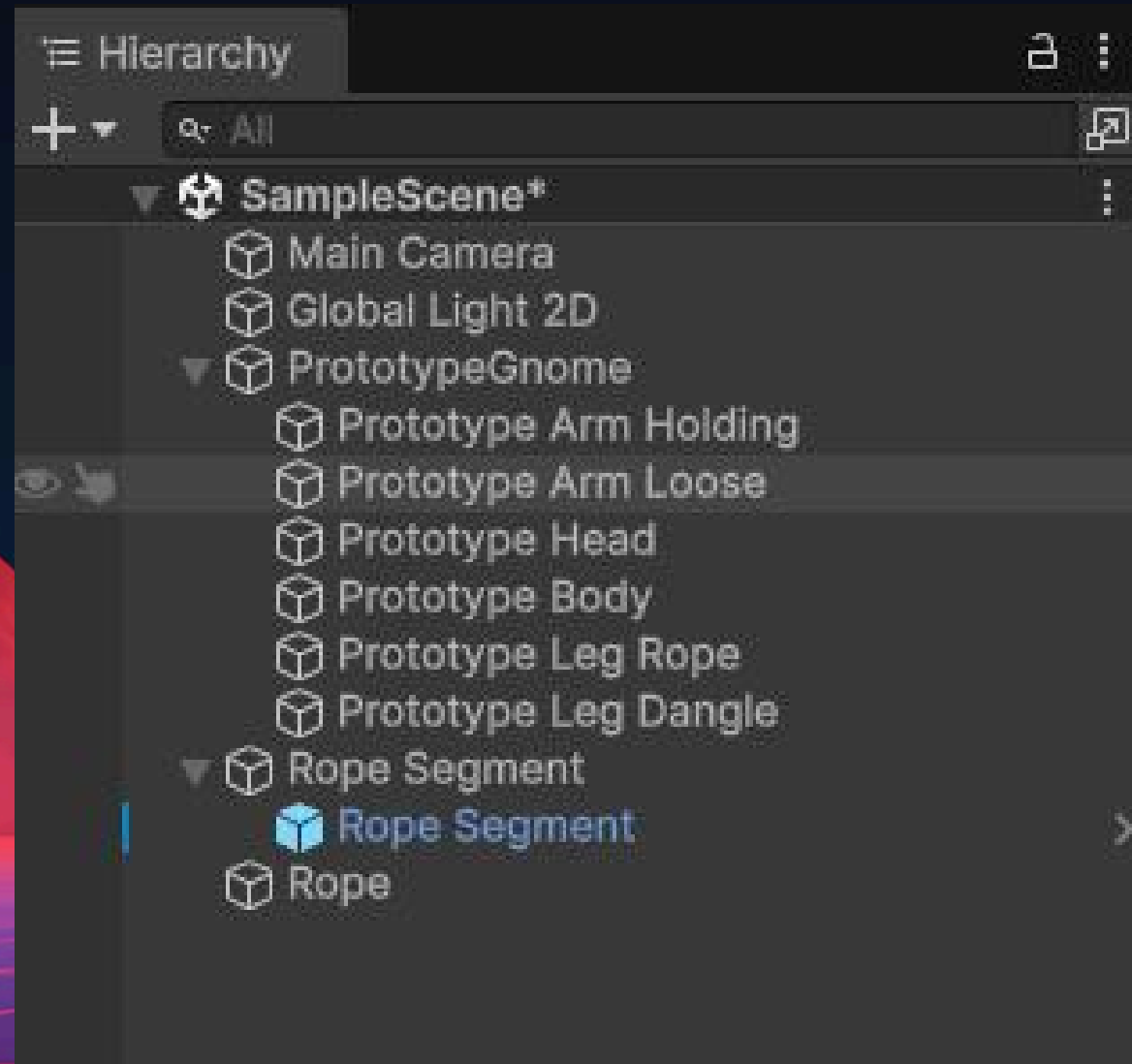
Add a component to each body parts

Add RigidBody2d, SpringJoint2d, HingeJoint2d, and add to the head circleCollider2d



STEP 5

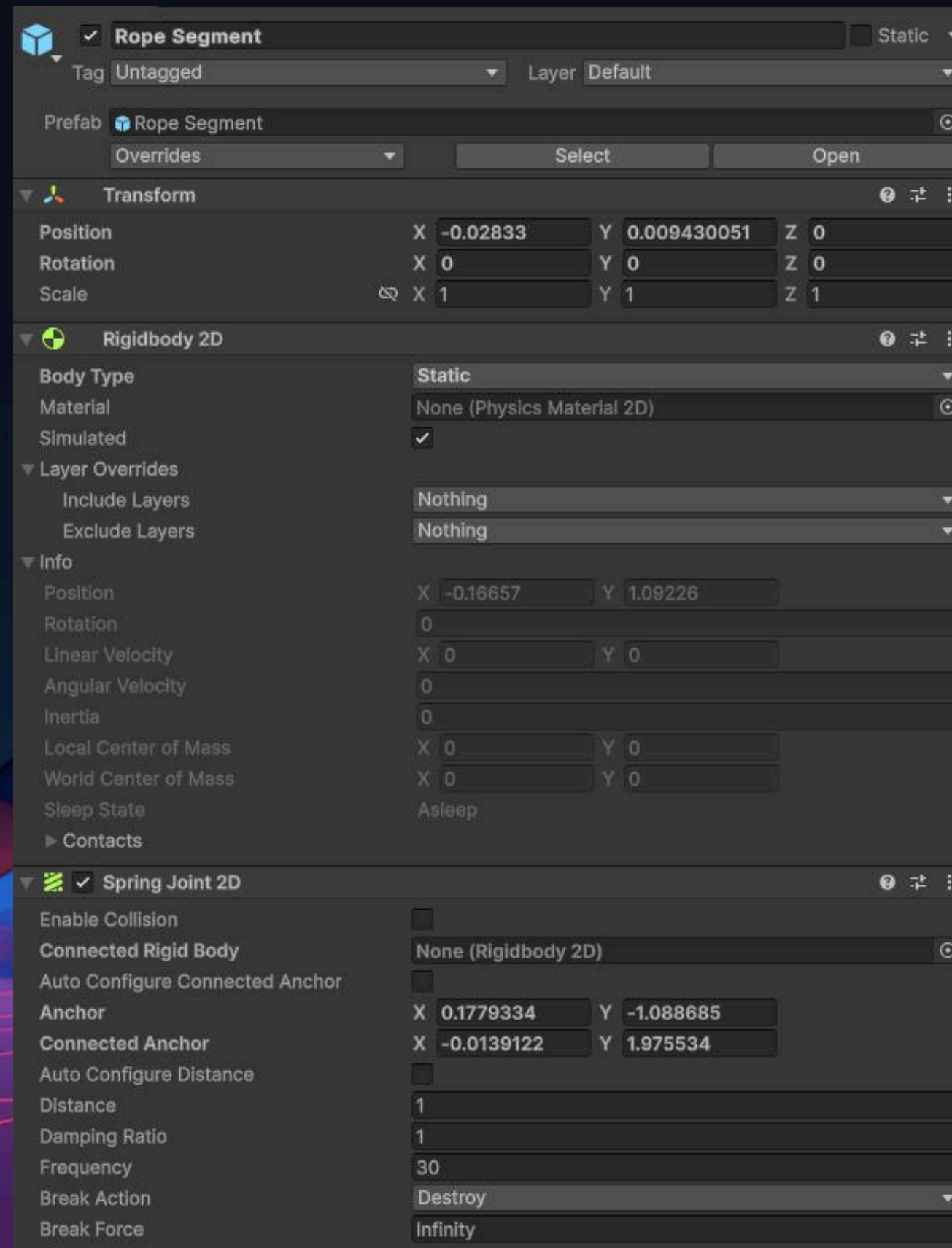
Create a empty game object name it
ROPE SEGMENT, and add the Rope
Segment prefab in the ROPE SEGMENT
you've created



STEP 6

Config the Rope Segment prefab,
change the value of the Damping Ratio
1, and Frequency to 30 of the SPRING
JOINT2D

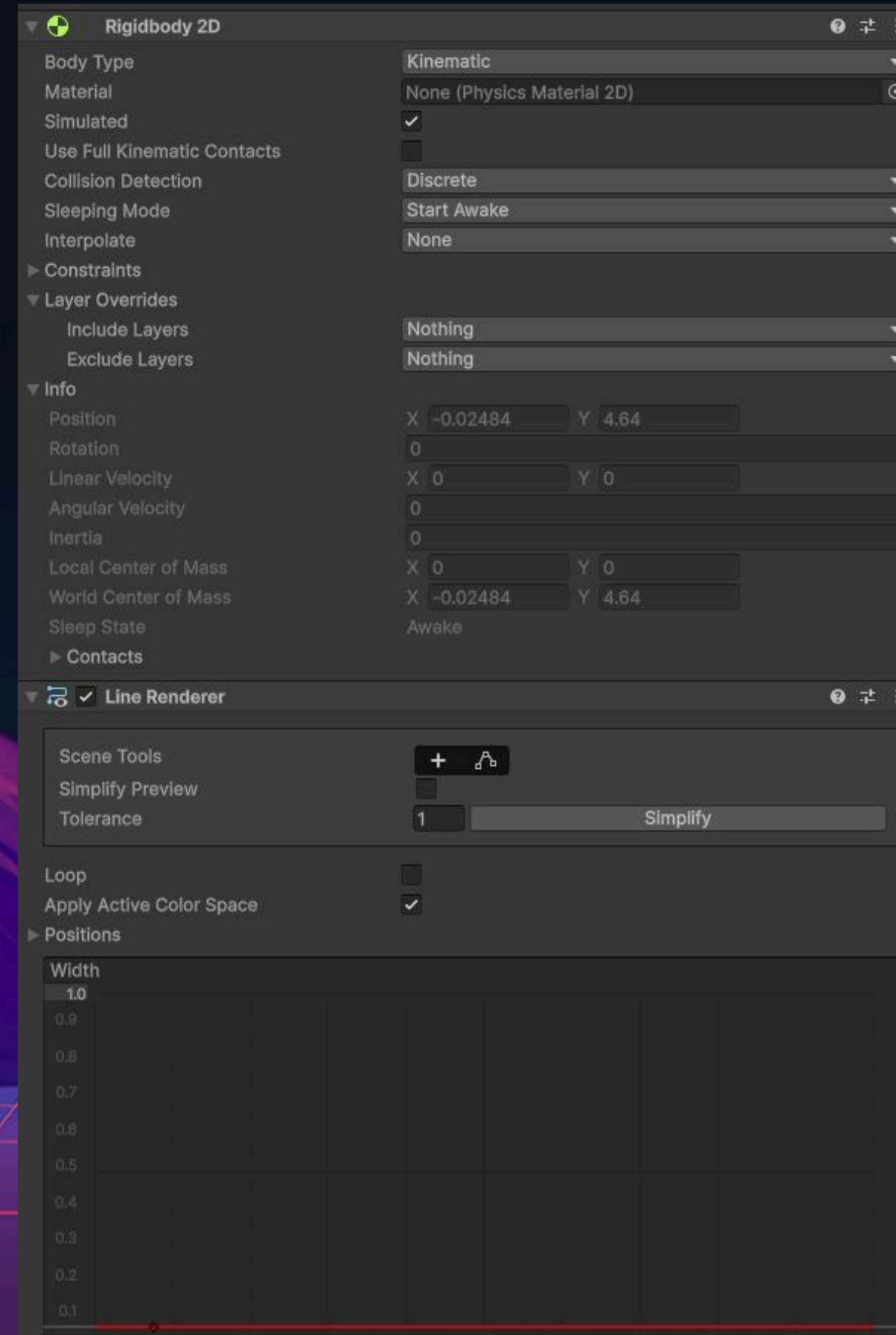
Change the body type of the
Rigidbody2D to STATIC so that the rope
segment will stay put



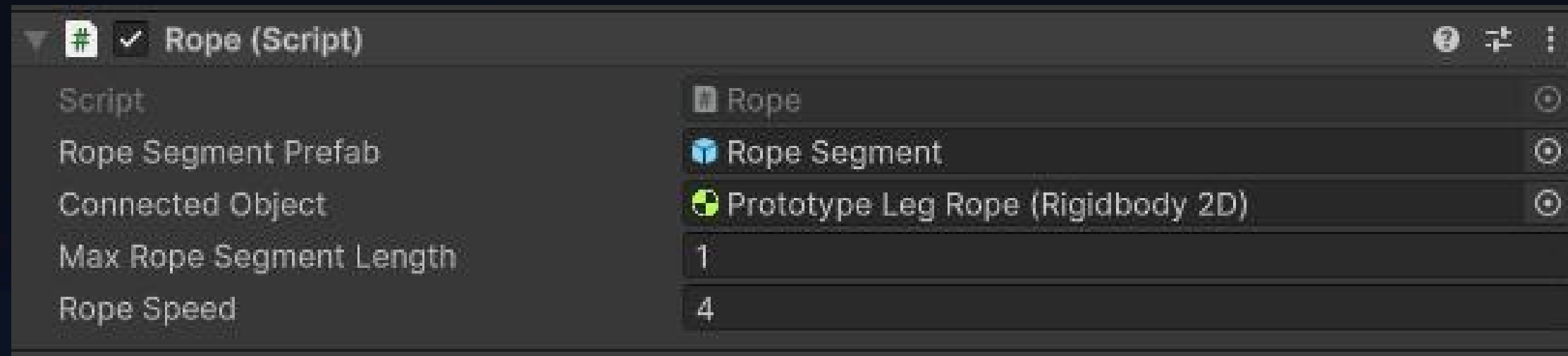
STEP 7

Add a component Rigidbody2d, change the body type to Kinematic so that it will move and stay the rope like a physics rope like a physics

Add a Line renderer which will create a block line to have a frame to the material to be added later, Change the width value to 0.0075 as the book recommended



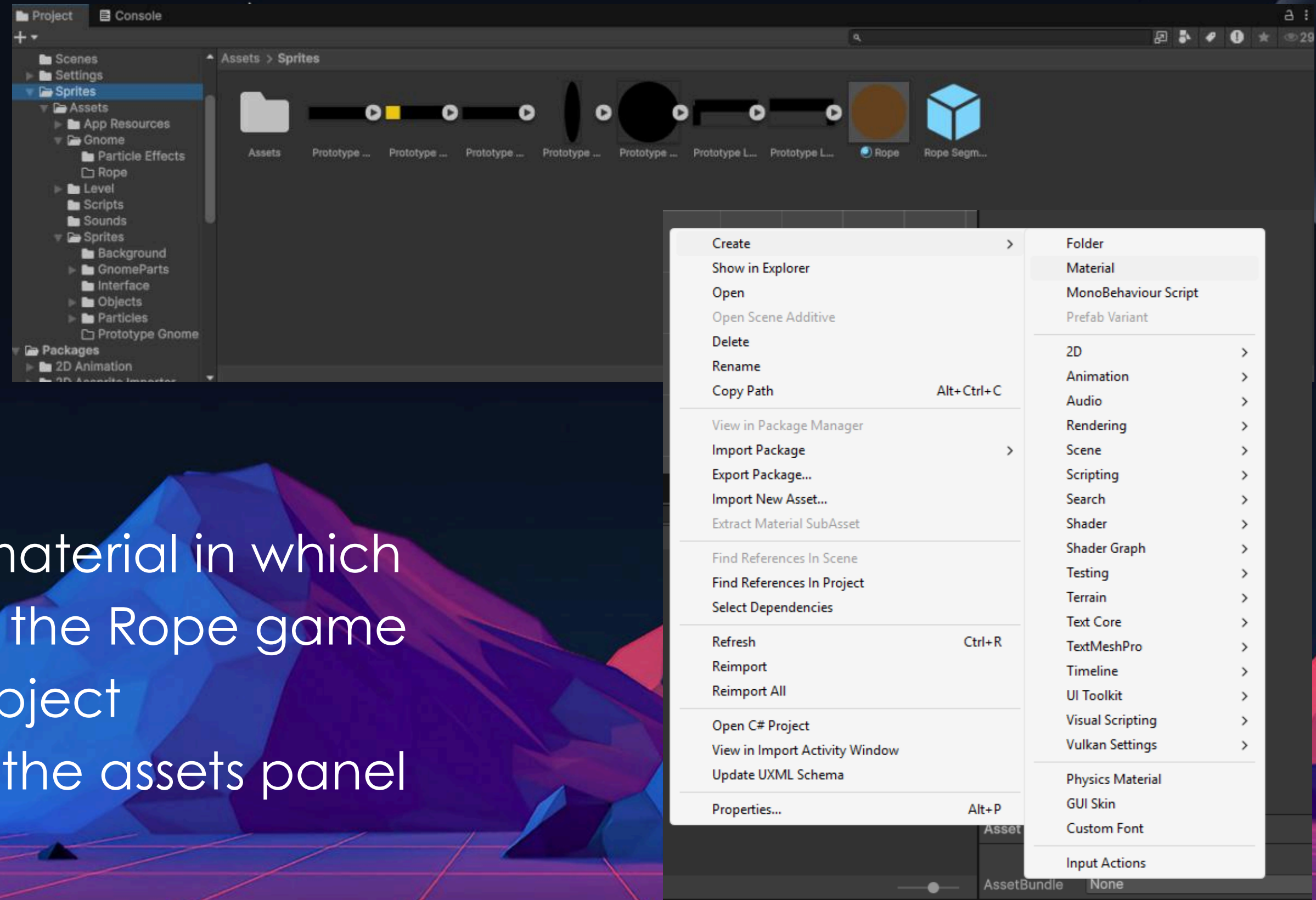
STEP 8



Add a new component called Rope.cs
note copy paste the code sa book

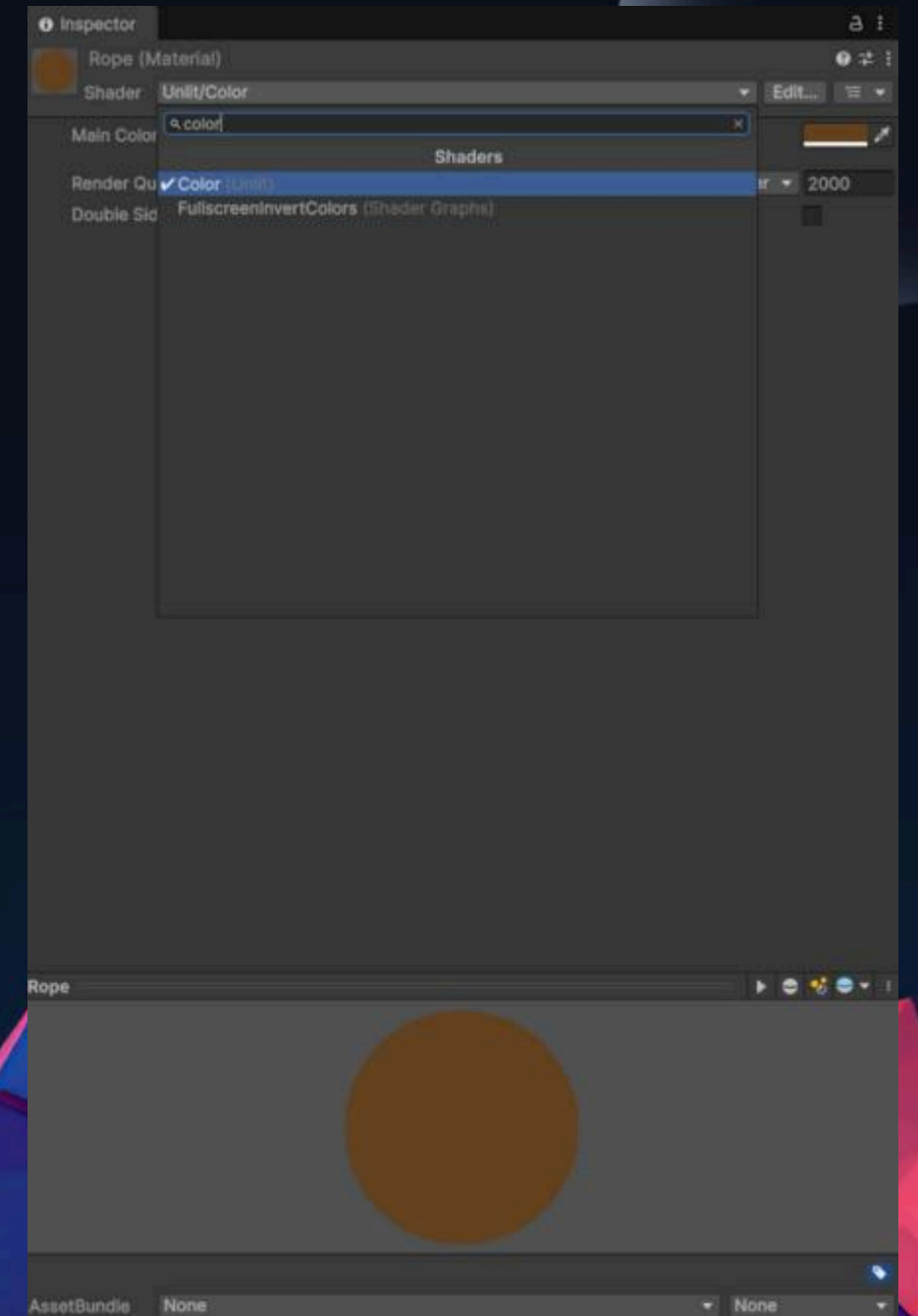
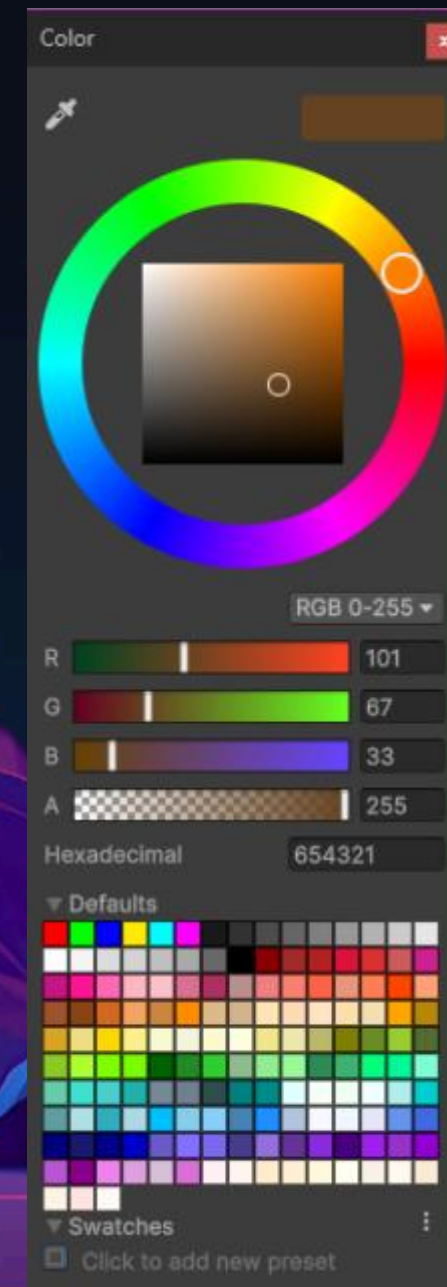
STEP 9

Add a new material in which
will be use to the Rope game
object
by right click the assets panel

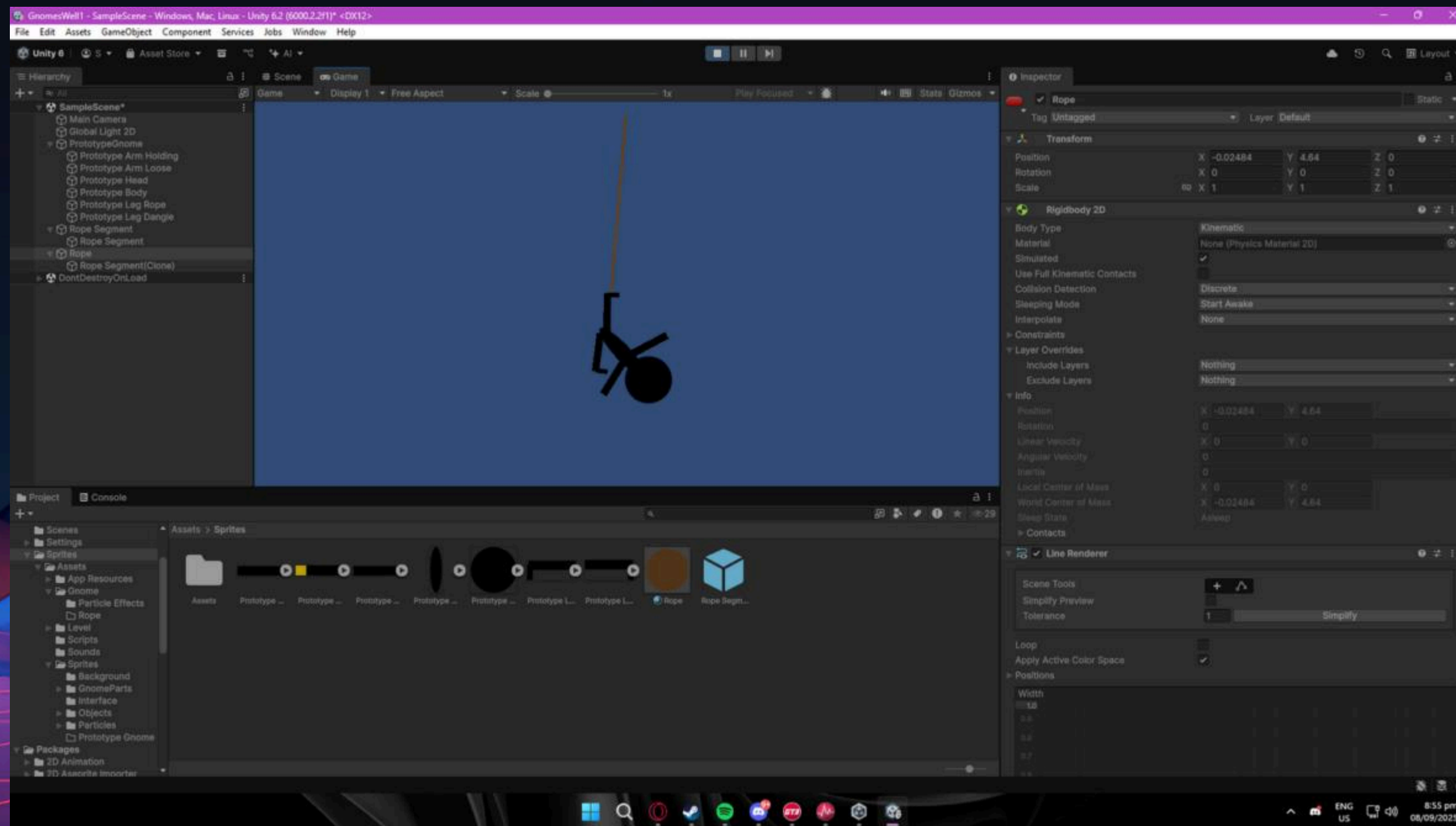


STEP 10

Set the Shader to Color and change the color to dark brown same the color of the rope we usually see



Game Prototype



Setting Up Buttons

Create another cs file names
singleton



```
Singleton (Mono Script)
Assembly Information
Filename Assembly-CSharp.dll

using UnityEngine;
using System.Collections;
// This class allows other objects to refer to a single
// shared object. The GameManager and InputManager
// classes
// use this.
// To use this, subclass like so:
// public class MyManager : Singleton<MyManager> { }
// You can then access the single shared instance of the
// class like so:
// MyManager.Instance.DoSomething();
public class Singleton<T> : MonoBehaviour
where T : MonoBehaviour
{
    // The single instance of this class.
    private static T _instance;
    // The accessor. The first time this is called, _instance
    // will be set up. If an appropriate object can't be
    found,
    // an error will be logged.
    public static T Instance
    {
        get
        {
            // If we haven't already set up _instance...
            if (_instance == null)
            {
                // Try to find the object.
                _instance = FindObjectOfType<T>();
                // Log if we can't find it.
                if (_instance == null)
                {
                    Debug.LogError("Can't find " +
                        typeof(T) + "!");
                }
            }
            // Return the instance so that it can be used!
            return _instance;
        }
    }
}
```

and add this code

Setting Up Buttons

Select all the asset body, and
create new cs paste the code

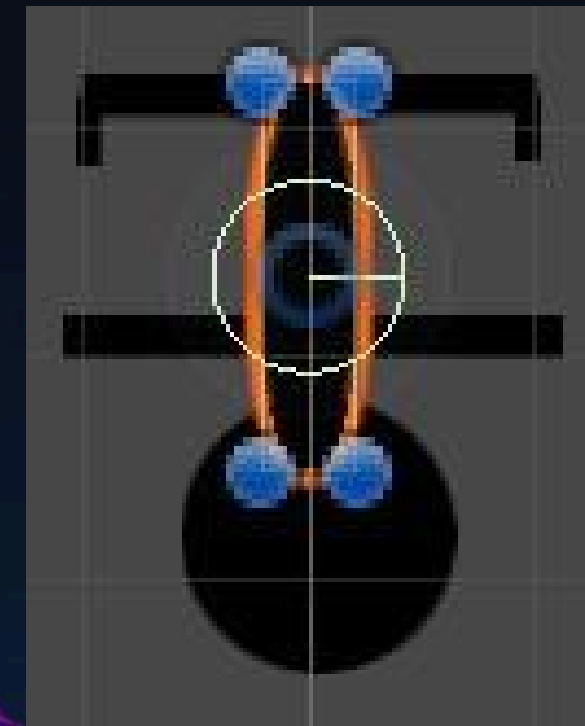
Create an object named
input manager, add the
script

```
Input Manager (Mono Script) Import Sett...
Open Execution Order...

Imported Object
# Input Manager (Mono Script)

Assembly Information
Filename Assembly-CSharp.dll

using UnityEngine;
using System.Collections;
// Translates the accelerometer data into sideways motion
// Info.
public class InputManager : Singleton<InputManager>
{
    // How much we're moving. -1.0 = full left, +1.0 = full
    // right
    private float _sidewaysMotion = 0.0f;
    // This property is declared as read-only, so that other
    // classes can't change it
    public float sidewaysMotion
    {
        get
        {
            return _sidewaysMotion;
        }
    }
    // Every frame, store the tilt
    void Update()
    {
        Vector3 accel = Input.acceleration;
        _sidewaysMotion = accel.x;
    }
}
```



```
Imported Object
# Swinging (Mono Script)

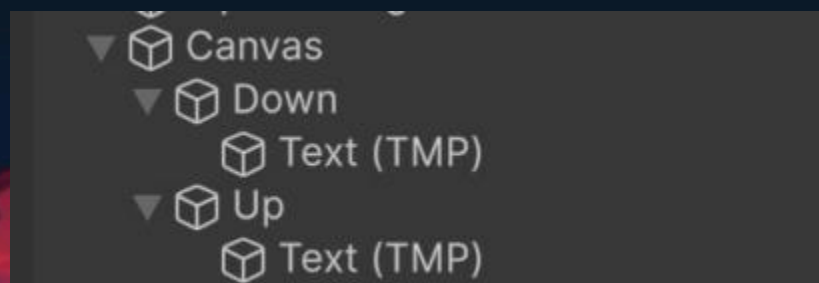
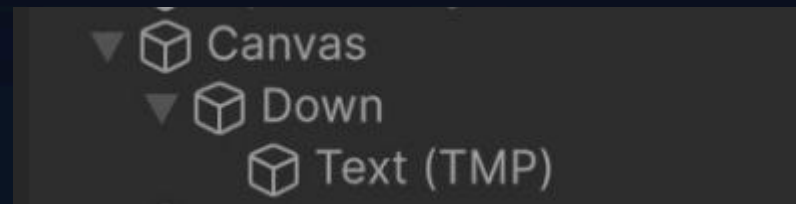
Assembly Information
Filename Assembly-CSharp.dll

using UnityEngine;
using System.Collections;
// Uses the input manager to apply sideways forces to an
// object. Used to make the gnome swing side-to-side.
public class Swinging : MonoBehaviour
{
    // How much should we swing by? Bigger numbers =
    // more
    // swing
    public float swingSensitivity = 100.0f;

    // Use FixedUpdate instead of Update, in order to play
    // better with the physics engine
    void FixedUpdate()
    {
        // If we have no rigidbody (anymore), remove this
        // component
        if (GetComponent<Rigidbody2D>() == null)
        {
            Destroy(this);
            return;
        }
        // Get the tilt amount from the InputManager
        float swing =
        InputManager.Instance.sidewaysMotion;
        // Calculate a force to apply
        Vector2 force =
        new Vector2(swing * swingSensitivity, 0);
        // Apply the force
        GetComponent<Rigidbody2D>().AddForce(force);
    }
}
```


Setting Up Buttons

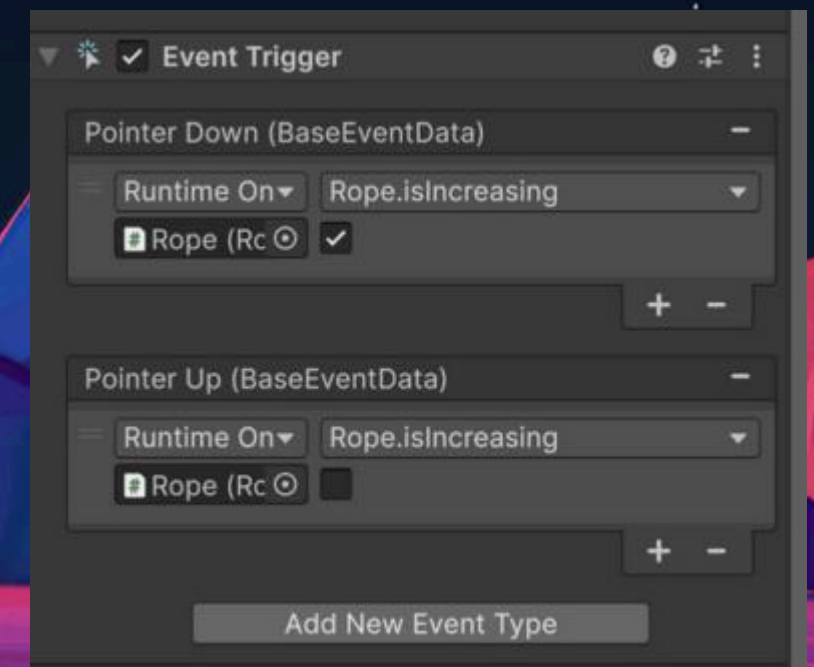
Add new game object and name it DOWN, Up



configure gameobject
(down,up)



Add Event Trigger component



Creating Camera Follow

Add the code cs
camerafollow.cs

```
1 using UnityEngine;
2
3 // Adjusts the camera to always match the Y-position of a
4 // target object, within certain limits.
5 public class CameraFollow : MonoBehaviour
6 {
7     // The object we want to match the Y position of.
8     public Transform target;
9     // The highest point the camera can go.
10    public float topLimit = 10.0f;
11    // The lowest point the camera can go.
12    public float bottomLimit = -10.0f;
13    // How quickly we should move toward the target.
14    public float followSpeed = 0.5f;
15    // After all objects have updated position, work out where
16    // this camera should be
17    void LateUpdate()
18    {
19        // If we have a target...
20        if (target != null)
21        {
22            // Get its position
23            Vector3 newPosition = this.transform.position;
24            // Work out where this camera should be
25            newPosition.y = Mathf.Lerp(newPosition.y,
26                                     target.position.y, followSpeed);
27            // Clamp this new location to within our
28            // limits
29            newPosition.y =
30                Mathf.Min(newPosition.y, topLimit);
31            newPosition.y =
32                Mathf.Max(newPosition.y, bottomLimit);
33            // Update our location
34            transform.position = newPosition;
35        }
36    }
37
38    // When selected in the editor, draw a line from the top
39    // limit to the bottom.
40    void OnDrawGizmosSelected()
41    {
42        Gizmos.color = Color.yellow;
43        Vector3 topPoint =
44            new Vector3(this.transform.position.x,
45                       topLimit, this.transform.position.z);
46        Vector3 bottomPoint =
47            new Vector3(this.transform.position.x,
48                       bottomLimit, this.transform.position.z);
49        Gizmos.DrawLine(topPoint, bottomPoint);
50    }
51 }
```


Creating Camera Follow

Add code cs Gnome.cs

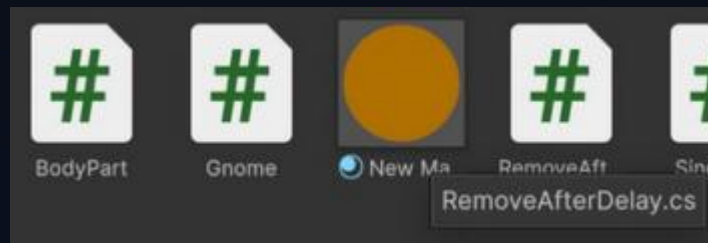
```
Gnome ▶ ReleaseGhost()

1 using System.Collections;
2 using UnityEngine;
3 public class Gnome : MonoBehaviour
4 {
5     // The object that the camera should follow.
6     public Transform cameraFollowTarget;
7     public Rigidbody2D ropeBody;
8     public Sprite armHoldingEmpty;
9     public Sprite armHoldingTreasure;
10    public SpriteRenderer holdingArm;
11    public GameObject deathPrefab;
12    public GameObject flameDeathPrefab;
13    public GameObject ghostPrefab;
14    public float delayBeforeRemoving = 3.0f;
15    public float delayBeforeReleasingGhost = 0.25f;
16    public GameObject bloodFountainPrefab;
17
18    bool dead = false;
19    bool _holdingTreasure = false;
20    public bool holdingTreasure
21    {
22        get
23        {
24            return _holdingTreasure;
25        }
26        set
27        {
28            if (dead == true)
29            {
30                return;
31            }
32            _holdingTreasure = value;
33            if (holdingArm != null)
34            {
35                if (_holdingTreasure)
36                {
37                    holdingArm.sprite = armHoldingTreasure;
38                }
39                else
40                {
41                    holdingArm.sprite = armHoldingEmpty;
42                }
43            }
44        }
45    }
46
47    public enum DamageType
48    {
49        Slicing,
50        Burning
51    }
52    public void ShowDamageEffect(DamageType type)
53    {
54        switch (type)
55        {
56            case DamageType.Burning:
57                if (flameDeathPrefab != null)
58                {
59                    Instantiate(
60                        flameDeathPrefab, cameraFollowTarget.position,
61                        cameraFollowTarget.rotation);
62                }
63                break;
64            case DamageType.Slicing:
65                if (deathPrefab != null)
66                {
67                    Instantiate(
68                        deathPrefab, cameraFollowTarget.position,
69                        cameraFollowTarget.rotation);
70                }
71                break;
72            }
73    }
74
75    public void DestroyGnome(DamageType type)
76    {
77    }
78
79    }
```

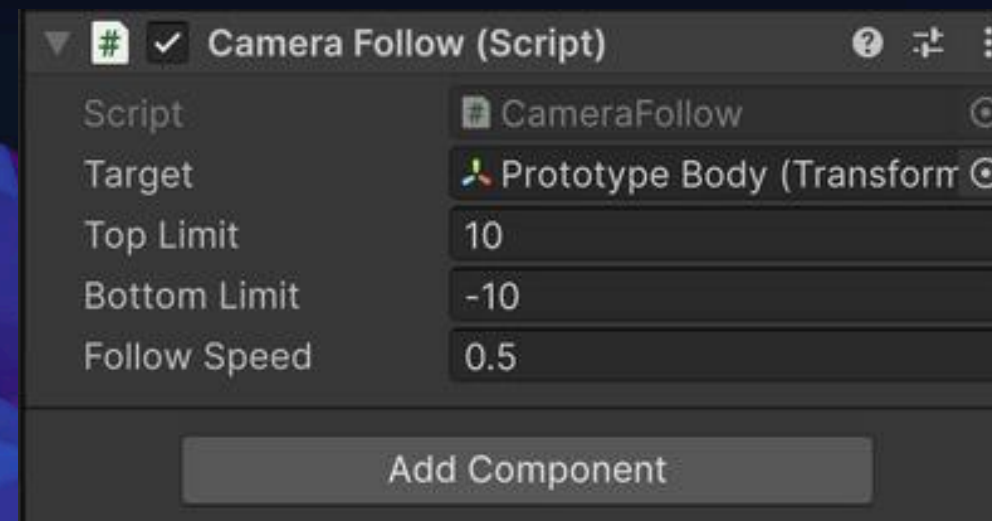
```
3
4 public void DestroyGnome(DamageType type)
5 {
6     holdingTreasure = false;
7     dead = true;
8     // Hide all child objects, and randomly disconnect
9     // their joints
10    foreach (BodyPart part in
11        GetComponentInChildrenBodyParts())
12    {
13        switch (type)
14        {
15            case DamageType.Burning:
16                // 1 in 3 chance of burning
17                bool shouldBurn = Random.Range(0, 2) == 0;
18                if (shouldBurn)
19                {
20                    part.ApplyDamageSprite(type);
21                }
22                break;
23            case DamageType.Slicing:
24                // Slice damage always applies a damage sprite
25                part.ApplyDamageSprite(type);
26                break;
27        }
28    }
29    // 2 in 3 chance of separating from body
30    bool shouldDetach = Random.Range(0, 2) == 0;
31    if (shouldDetach)
32    {
33        // Make this object remove its rigidbody and
34        // collider after it comes to rest
35        part.Detach();
36        // If we're separating, and the damage type was
37        // Slicing, add a blood fountain
38        if (type == DamageType.Slicing)
39        {
40            if (part.bloodFountainBrigun != null &&
41                bloodFountainPrefab != null)
42            {
43                // Attach a blood fountain for
44                // this detached part
45                GameObject fountain = Instantiate(
46                    bloodFountainPrefab,
47                    part.bloodFountainBrigun.position,
48                    part.bloodFountainBrigun.rotation,
49                    this.gameObject);
50                fountain.transform.SetParent(
51                    this.cameraFollowTarget,
52                    false);
53            }
54        }
55        // Disconnect this object
56        var allJoints = part.GetComponentInChildren<Joint2D>();
57        foreach (Joint2D joint in allJoints)
58        {
59            Destroy(joint);
60        }
61    }
62
63    // Add a RemoveAfterDelay component to this object
64    var remove = gameObject.AddComponent<RemoveAfterDelay>();
65    remove.Delay = delayBeforeRemoving;
66    StartCoroutine(RemoveGhost());
67
68    IEnumerator ReleaseGhost()
69    {
70        // No ghost prefab? Bail out.
71        if (ghostPrefab == null)
72        {
73            yield break;
74        }
75        // Wait for delayBeforeReleasingGhost seconds
76        yield return new WaitForSeconds(delayBeforeReleasingGhost);
77        // Add the ghost
78        Instantiate(
79            ghostPrefab,
80            transform.position,
81            Quaternion.identity);
82    }
83
84    }
```


Creating Camera Follow

create 3 cs files BodyPart,Gnome
and Removeafter delay



Add Camera Follow script on the main
camera object and Drag prototype
body in the target



Add to the BodyPart

```
1 using UnityEngine;
2 [RequireComponent(typeof(SpriteRenderer))]
3 public class BodyPart : MonoBehaviour
4 {
5     // The sprite to use when ApplyDamageSprite is called with
6     // damage type 'slicing'
7     public Sprite detachedSprite;
8     // The sprite to use when ApplyDamageSprite is called with
9     // damage type 'burning'
10    public Sprite burnedSprite;
11    // Represents the position and rotation that a blood
12    // fountain will appear at on the main body
13    public Transform bloodFountainOrigin;
14    // If true, this object will remove its collision, joints,
15    // and rigidbody when it comes to rest
16    bool detached = false;
17    // Decouple this object from the parent, and flag it as
18    // needing physics removal
19    public void Detach()
20    {
21        detached = true;
22        this.tag = "Untagged";
23        transform.SetParent(null, true);
24    }
25
26    // Every frame, if we're detached, remove physics if the
27    // rigidbody is sleeping. This means this detached body
28    // part will never get in the way of the gnome.
29    public void Update()
30    {
31        // If we're not detached, do nothing
32        if (detached == false)
33        {
34            return;
35        }
36        // Is our rigidbody sleeping?
37        var rigidbody = GetComponent<Rigidbody2D>();
38        if (rigidbody.IsSleeping())
39        {
40            // If so, destroy all joints..
41            foreach (Joint2D joint in
42                GetComponentsInChildren<Joint2D>())
43            {
44                Destroy(joint);
45            }
46            // ...rigidbodies...
47            foreach (Rigidbody2D body in
48                GetComponentsInChildren<Rigidbody2D>())
49            {
50                Destroy(body);
51            }
52            // ...and the collider.
53            foreach (Collider2D collider in
54                GetComponentsInChildren<Collider2D>())
55            {
56                Destroy(collider);
57            }
58            // Finally, remove this script.
59            Destroy(this);
60        }
61    }
62
63    // Swaps out the sprite for this part based on what kind
64    // of damage was received
65    public void ApplyDamageSprite(
66        Gnome.DamageType damageType)
67    {
68        Sprite spriteToUse = null;
69        switch (damageType)
70        {
71            case Gnome.DamageType.Burning:
72                spriteToUse = burnedSprite;
73                break;
74            case Gnome.DamageType.Slicing:
75                spriteToUse = detachedSprite;
76                break;
77        }
78        if (spriteToUse != null)
79        {
80            GetComponent<SpriteRenderer>().sprite =
81                spriteToUse;
82        }
83    }
84 }
```


Adjusting game parameters

Add this to the RemoveAfterDelay.cs

Select all the body parts and after that Prototype gnomewell too put the BodyPart.cs

Configured Gnome.cs looks like

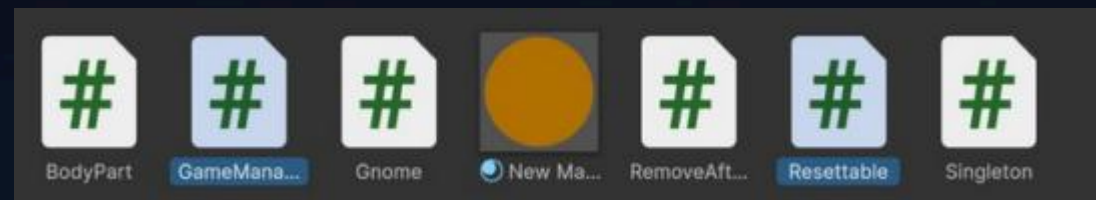
```
RemoveAfterDelay ▶ Remove()
1 // Removes an object after a certain delay.
2 using UnityEngine;
3 using System.Collections;
4 public class RemoveAfterDelay : MonoBehaviour
5 {
6     // How many seconds to wait before removing.
7     public float delay = 1.0f;
8     void Start()
9     {
10         // Kick off the 'Remove' coroutine.
11         StartCoroutine("Remove");
12     }
13     IEnumerator Remove()
14     {
15         // Wait 'delay' seconds, and then destroy the
16         // gameObject attached to this object.
17         yield return new WaitForSeconds(delay);
18         Destroy(gameObject);
19         // Don't say Destroy(this) - that just destroys this
20         // RemoveAfterDelay script.
21     }
22 }
```

Prototype Arm Holding
Prototype Arm Loose
Prototype Body
Prototype Head
Prototype Leg Dangle
Prototype Leg Rope

Gnome (Script)		
Script	Gnome	
Camera Follow Tar...	Prototype Body (Transform)	
Rope Body	Prototype Leg Rope (Rigidbody)	
Arm Holding Empty	Prototype Arm Holding	
Arm Holding Treas...	Prototype Arm Holding with C	
Holding Arm	Prototype Arm Holding (Sprite)	
Death Prefab	None (Game Object)	
Flame Death Prefab	None (Game Object)	
Ghost Prefab	None (Game Object)	
Delay Before Rem...	3	
Delay Before Rele...	0.25	
Blood Fountain Pre...	None (Game Object)	
Add Component		

Adjusting game parameters

create new 2 file cs named
gamemanager and resettable



copy paste the code
from the book

```
1 using UnityEngine;
2 using UnityEngine.Events;
3 // Contains a UnityEvent that can be used to reset the state
4 // of this object.
5 public class Resetable : MonoBehaviour
6 {
7     // In the editor, connect this event to methods that should
8     // run when the game resets.
9     public UnityEvent onReset;
10    // Called by the GameManager when the game resets.
11    public void Reset()
12    {
13        // Kicks off the event, which calls all of the
14        // connected methods.
15        onReset.Invoke();
16    }
17 }
```


Adjusting game parameters

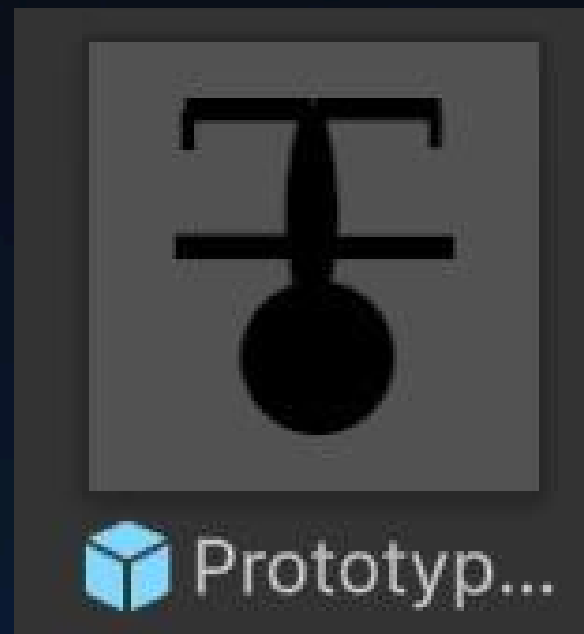
Add this code to the game manager from the book

```
1 using System.Collections;
2 using UnityEngine;
3
4 // Manages the game state.
5 public class GameManager : Singleton<GameManager>
6 {
7     // The location where the gnome should appear.
8     public GameObject startingPoint;
9     // The rope object, which lowers and raises the gnome.
10
11     public Rope rope;
12     // The follow script, which will follow the gnome
13     public CameraFollow cameraFollow;
14     // The 'current' gnome (as opposed to all those dead ones)
15     Gnome currentGnome;
16     // The prefab to instantiate when we need a new gnome
17     public GameObject gnomePrefab;
18     // The UI component that contains the 'restart' and 'resume'
19     // buttons
20     public RectTransform mainMenu;
21     // The UI component that contains the 'up', 'down' and
22     // 'menu' buttons
23     public RectTransform gameplayMenu;
24     // The UI component that contains the 'you win!' screen
25     public RectTransform gameOverMenu;
26     // If true, ignore all damage (but still show damage
27     // effects) The 'get; set;' make this a property, to make
28     // it show up in the list of methods in the Inspector for
29     // Unity Events
30     public bool gnomeInvincible { get; set; }
31     // How long to wait after dying before creating a new gnome
32     public float delayAfterDeath = 1.0f;
33     // The sound to play when the gnome dies
34     public AudioClip gnomeDiedSound;
35     // The sound to play when the game is won
36     public AudioClip gameOverSound;
37     void Start()
38     {
39         // When the game starts, call Reset to set up the
40         // gnome.
41         Reset();
42     }
43     // Reset the entire game.
44     public void Reset()
45     {
46         // Turn off the menus, turn on the gameplay UI
47         if (gameOverMenu)
48             gameOverMenu.gameObject.SetActive(false);
49         if (mainMenu)
50             mainMenu.gameObject.SetActive(false);
51         if (gameplayMenu)
52             gameplayMenu.gameObject.SetActive(true);
53         // Find all Resettable components and tell them to
```

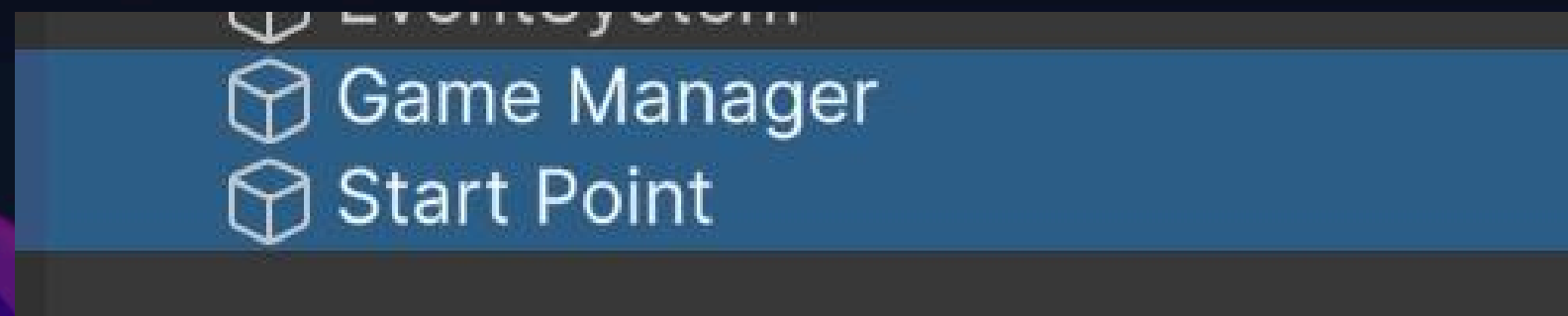
```
64 // Un-pause the game
65 Time.timeScale = 1.0f;
66 }
67 void CreateNewGnome()
68 {
69     // Remove the current gnome, if there is one
70     RemoveGnome();
71     // Create a new Gnome object, and make it be our
72     // currentGnome
73     GameObject newGnome =
74         (GameObject)Instantiate(gnomePrefab,
75             startingPoint.transform.position,
76             Quaternion.identity);
77     currentGnome = newGnome.GetComponent<Gnome>();
78     // Make the rope visible
79     rope.gameObject.SetActive(true);
80     // Connect the rope's trailing end to whichever
81     // rigidbody the Gnome object wants (e.g., his foot)
82     rope.connectedObject = currentGnome.ropeBody;
83     // Reset the rope's length to the default
84     rope.ResetLength();
85     // Tell the cameraFollow to start tracking the new
86     // Gnome object
87     cameraFollow.target = currentGnome.cameraFollowTarget;
88 }
89 void RemoveGnome()
90 {
91     // Don't actually do anything if the gnome is invincible
92     if (gnomeInvincible)
93         return;
94     // Hide the rope
95     rope.gameObject.SetActive(false);
96     // Stop tracking the gnome
97     cameraFollow.target = null;
98     // If we have a current gnome, make that no longer be
99     // the player
100     if (currentGnome != null)
101     {
102         // This gnome is no longer holding the treasure
103         currentGnome.holdingTreasure = false;
104         // Mark this object as not the player (so that
105         // colliders won't report when the object
106         // hits them)
107         currentGnome.gameObject.tag = "Untagged";
108         // Find everything that's currently tagged
109         // "Player" and remove that tag
110         foreach (Transform child in
111             currentGnome.transform)
112         {
113             child.gameObject.tag = "Untagged";
114         }
115     }
116 }
```


Modifying the Gnome

make the gnome prototype into prefab

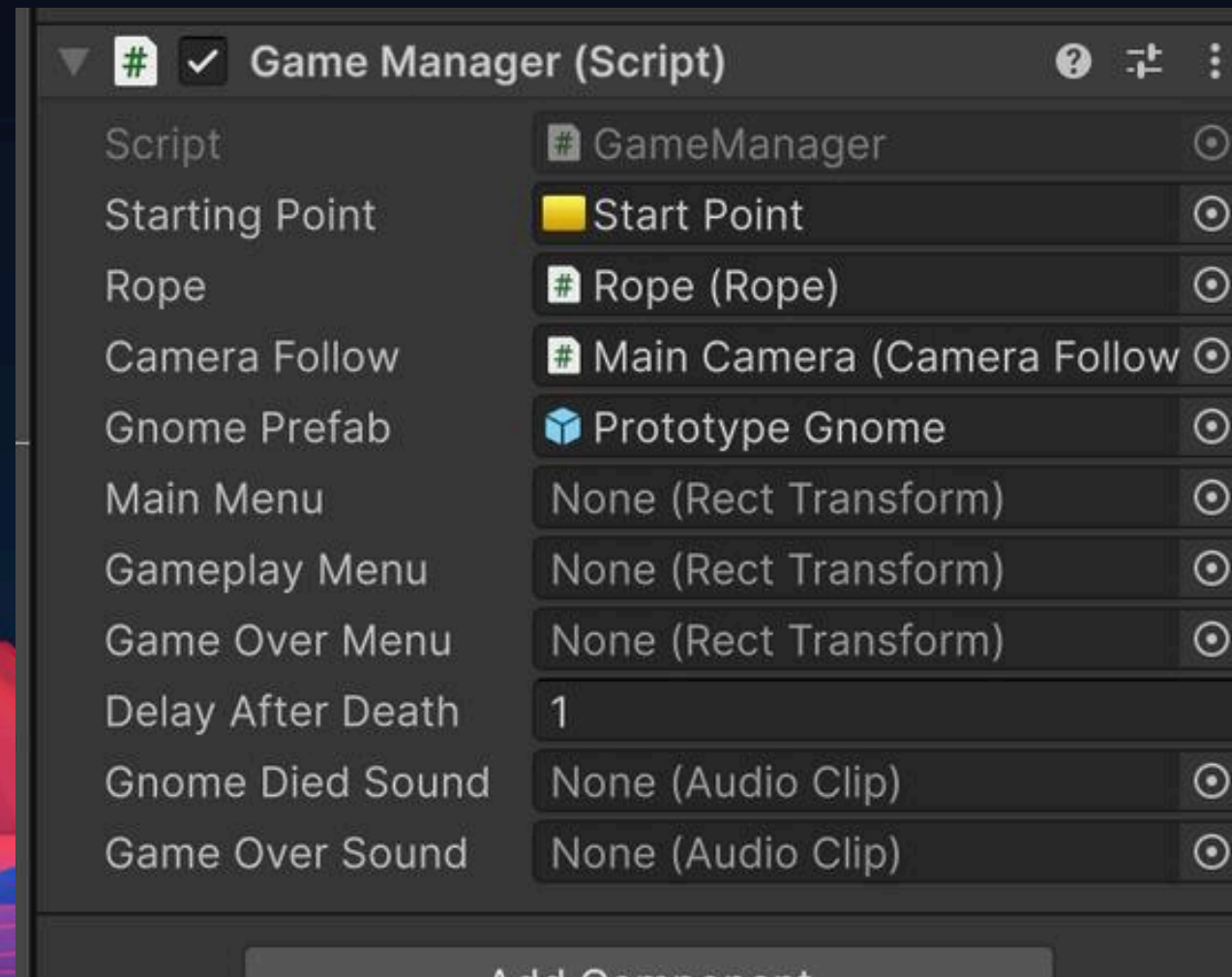


Create new gameobject and name it
Gmane Manager and Start point



Modifying the Gnome

Put the `GameManager.cs` into the game manager object and configure this setting given



Align Start point object to the gnome axis



Modifying the Gnome

add the sprites and replace the prefab with the actual sprites



Add new component collider object to each part except the body and inside the collider add polygon collider will look like this

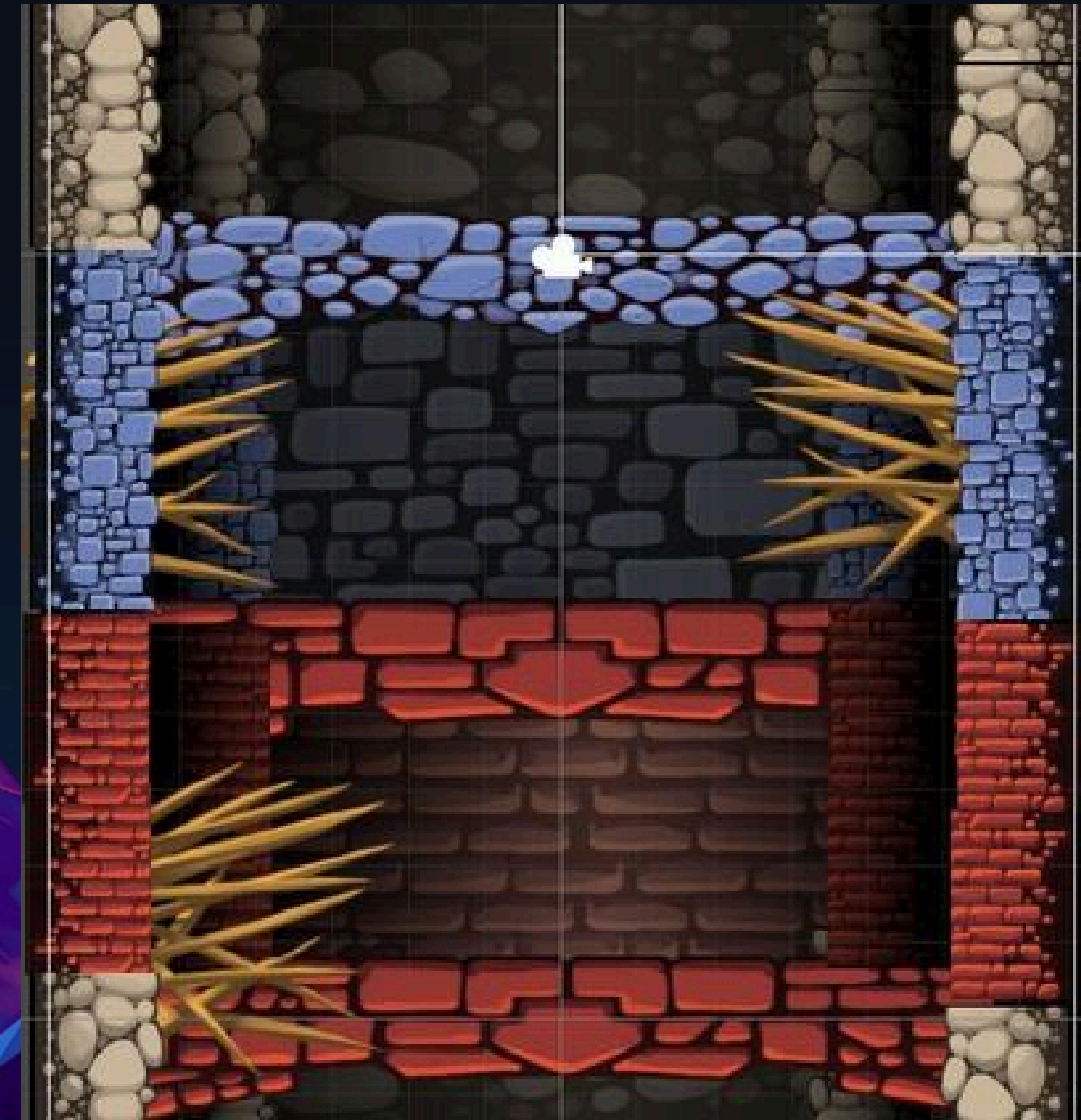
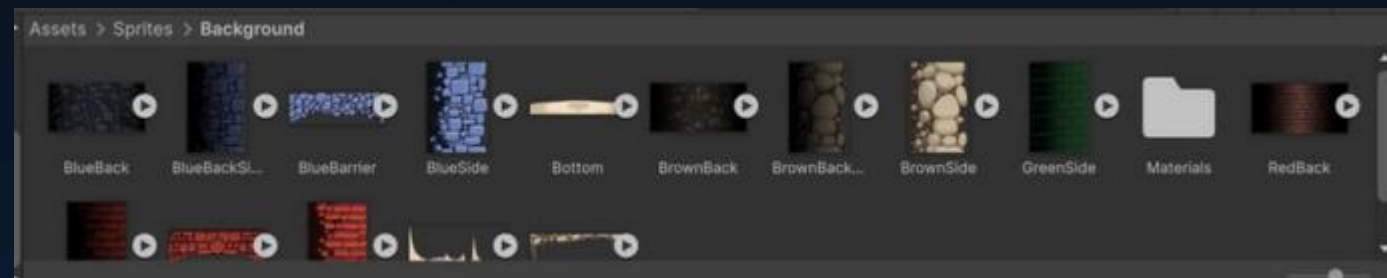
- ▼ Prototype Arm Holding
Collider
- ▼ Prototype Arm Loose
Collider
- Prototype Body
- ▼ Prototype Head
Collider
- ▼ Prototype Leg Dangle
Collider
- ▼ Prototype Leg Rope
Collider



Designing the Level

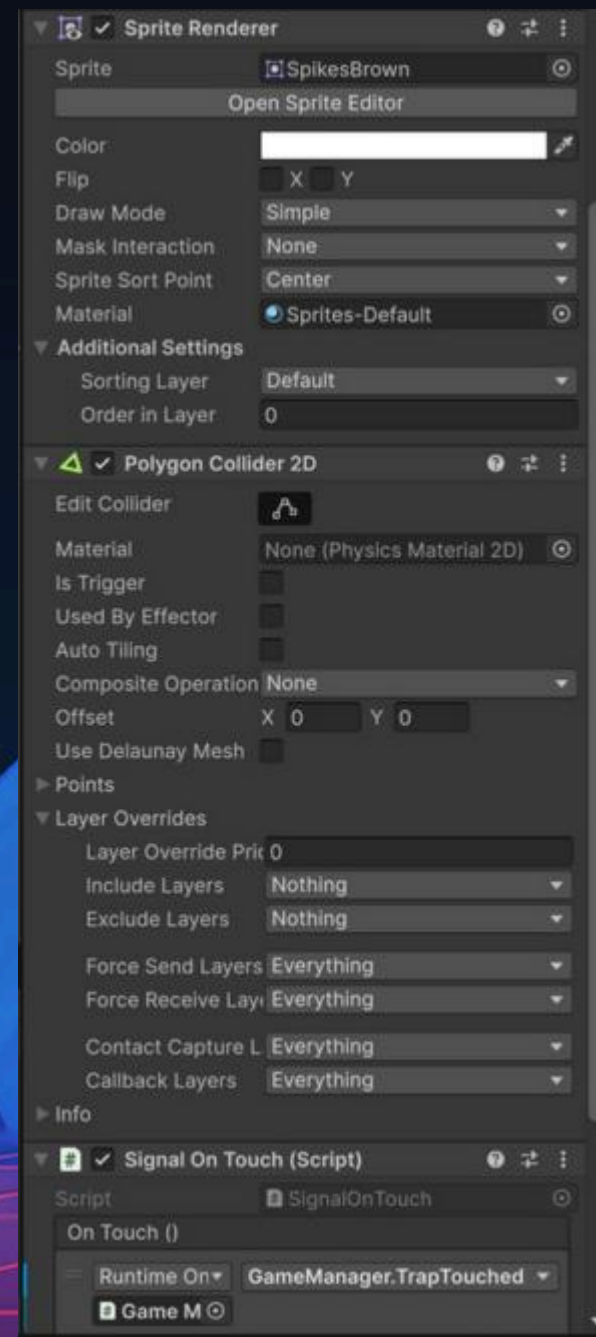
Do want you want, design
the level on your own

Add the asset for the level



Designing the Level

Add this components to the spikes



create new cs again paste the code and name it signalontouch.cs

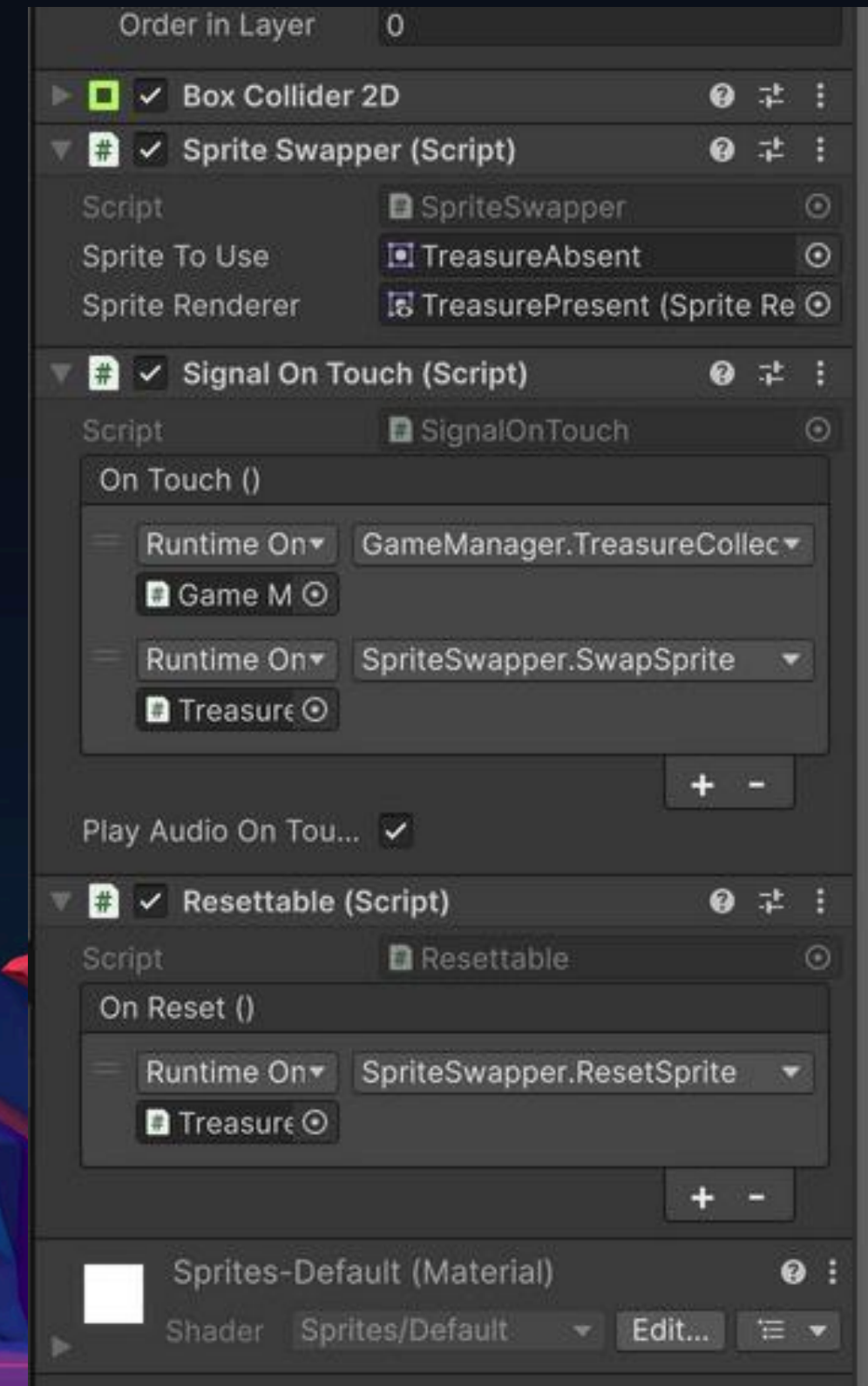
```
1
2 using UnityEngine;
3 using UnityEngine.Events;
4 // Invokes a UnityEvent when the Player collides with this
5 // object.
6 [RequireComponent(typeof(Collider2D))]
7 public class SignalOnTouch : MonoBehaviour
8 {
9
10 // The UnityEvent to run when we collide.
11 // Attach methods to run in the editor.
12 public UnityEvent onTouch;
13
14 // If true, attempt to play an AudioSource when we collide.
15 public bool playAudioOnTouch = true;
16 // When we enter a trigger area, call SendSignal.
17 void OnTriggerEnter2D(Collider2D collider)
18 {
19     SendSignal(collider.gameObject);
20 }
21 // When we collide with this object, call SendSignal.
22 void OnCollisionEnter2D(Collision2D collision)
23 {
24     SendSignal(collision.gameObject);
25 }
26 // Checks to see if this object was tagged as Player, and
27 // invoke the UnityEvent if it was.
28 void SendSignal(GameObject objectThatHit)
29 {
30     // Was this object tagged Player?
31     if (objectThatHit.CompareTag("Player"))
32     {
33         // If we should play a sound, attempt to play it
34         if (playAudioOnTouch)
35         {
36             var audio = GetComponent();
37             // If we have an audio component,
38             // and this component's parents
39             // are active, then play
40             if (audio &&
41                 audio.gameObject.activeInHierarchy)
42                 audio.Play();
43         }
44         // Invoke the event
45         onTouch.Invoke();
46     }
47 }
```


Designing the Level

and create the treasure add
from the asset

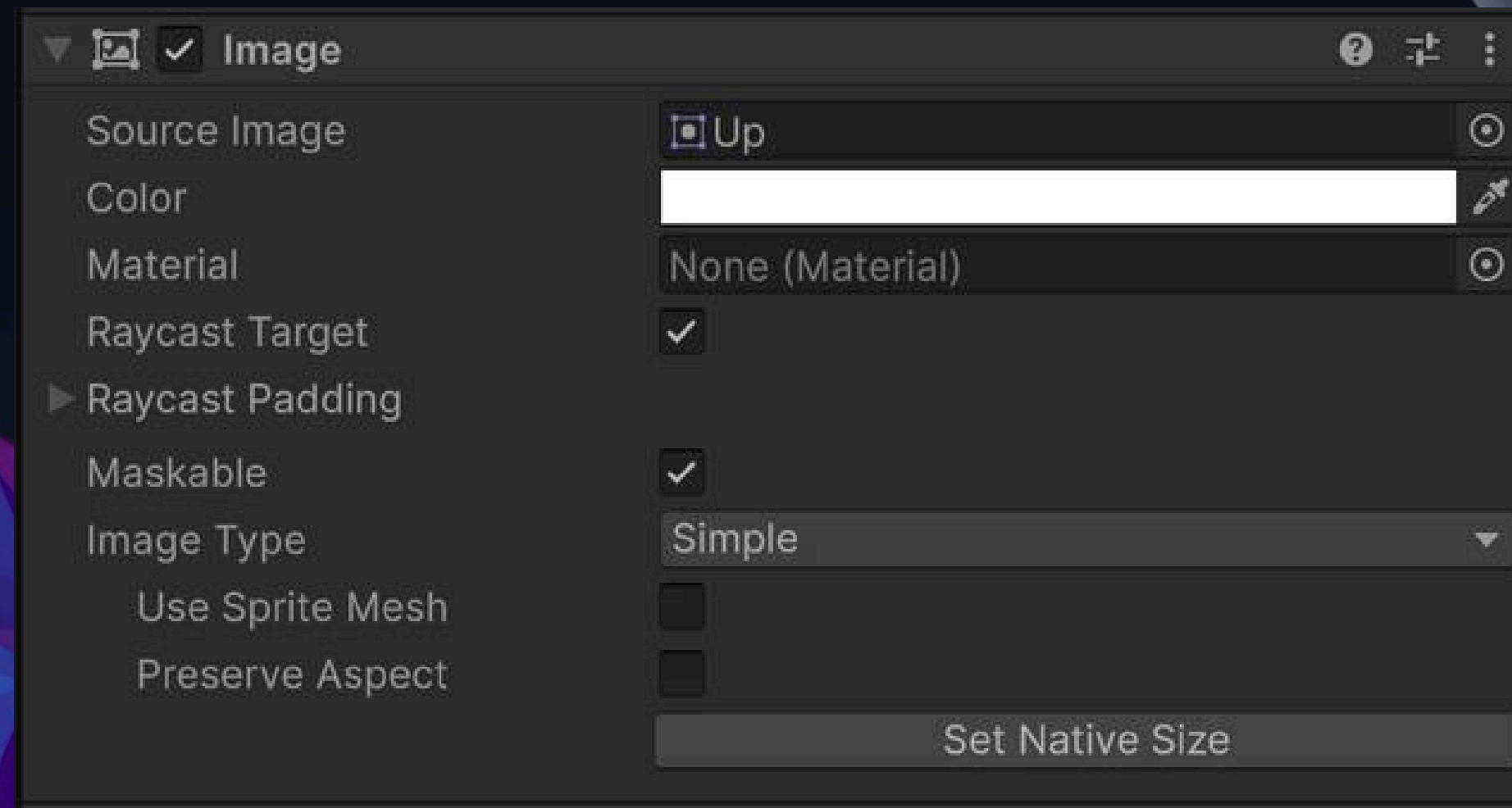


add component to the treasure



Setting Up Buttons

Click the 2 up, down
gameobject button and
remove the text and add
this component and add
the image from the asset
folder

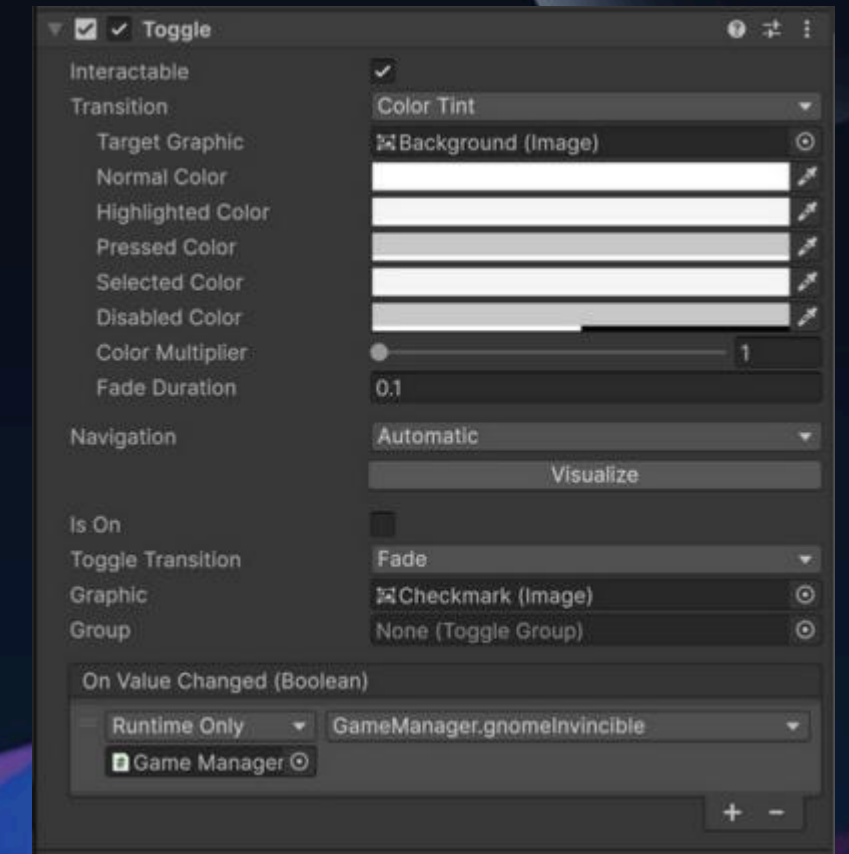
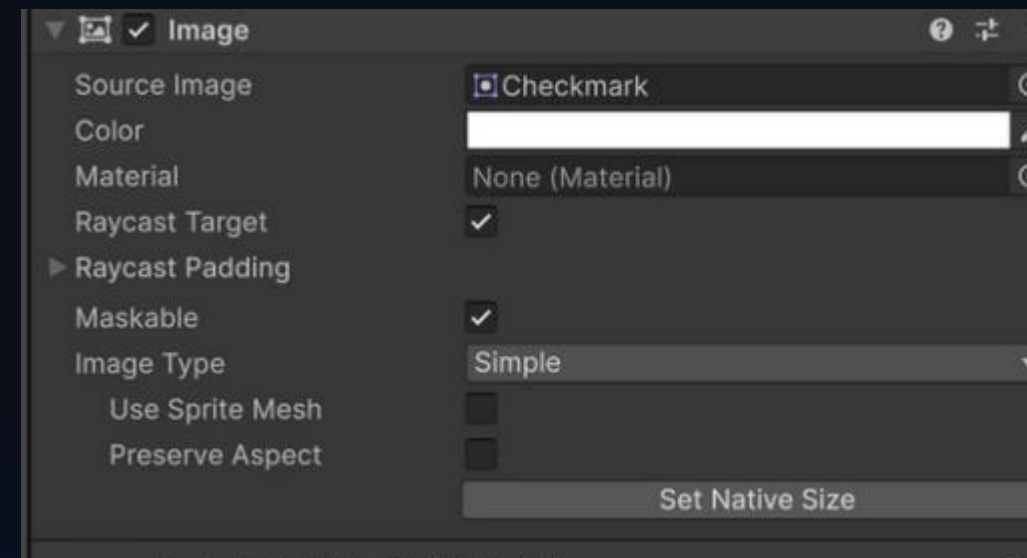
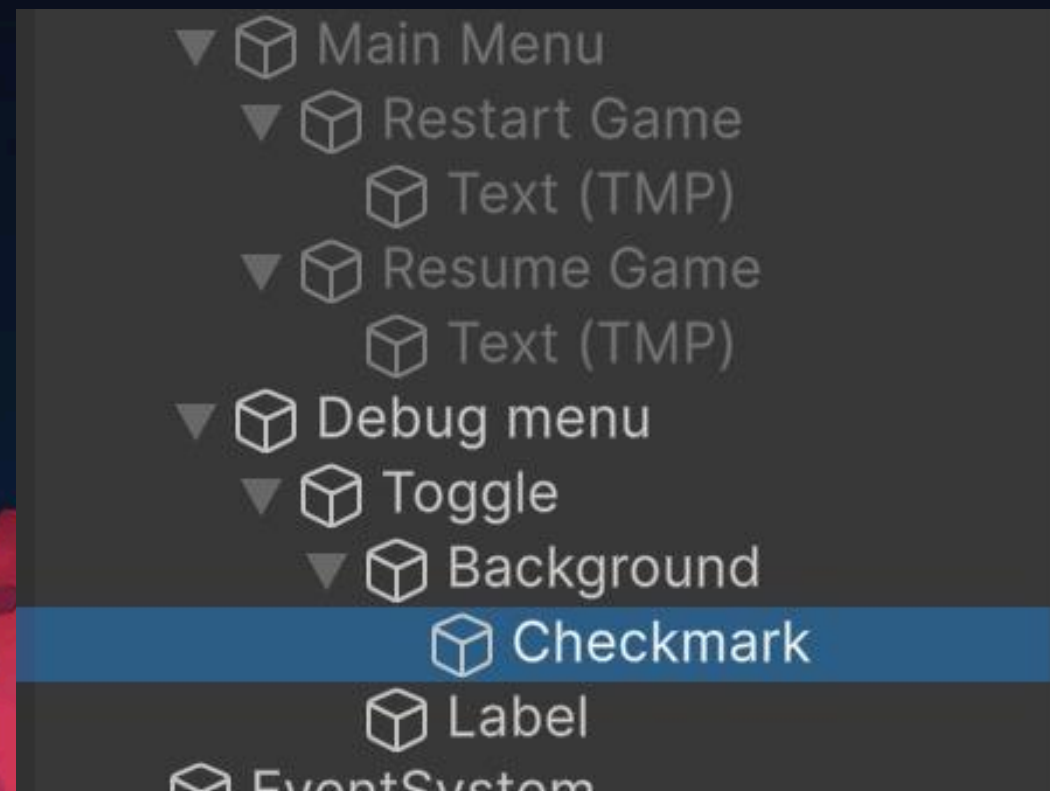


Game Mechanics

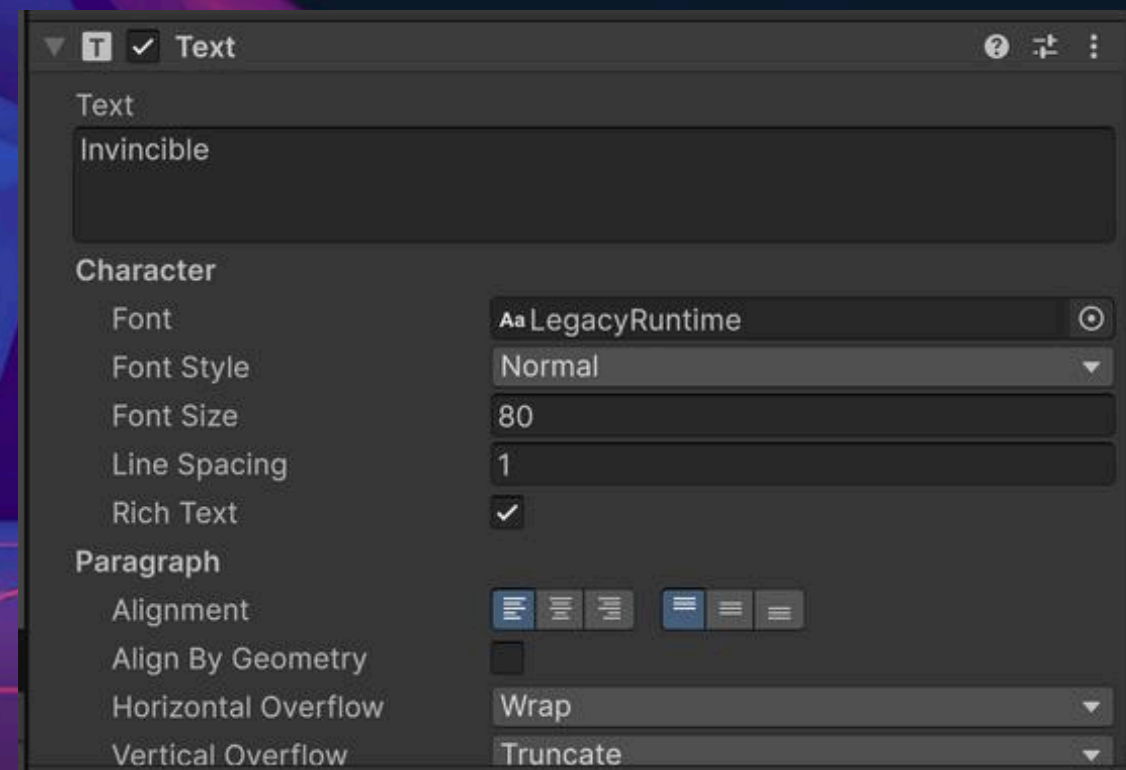
Add tis on the checkmark
object

And put this on the toggle
object

Add menu, invincible button and
make the objects with parent
and child objects

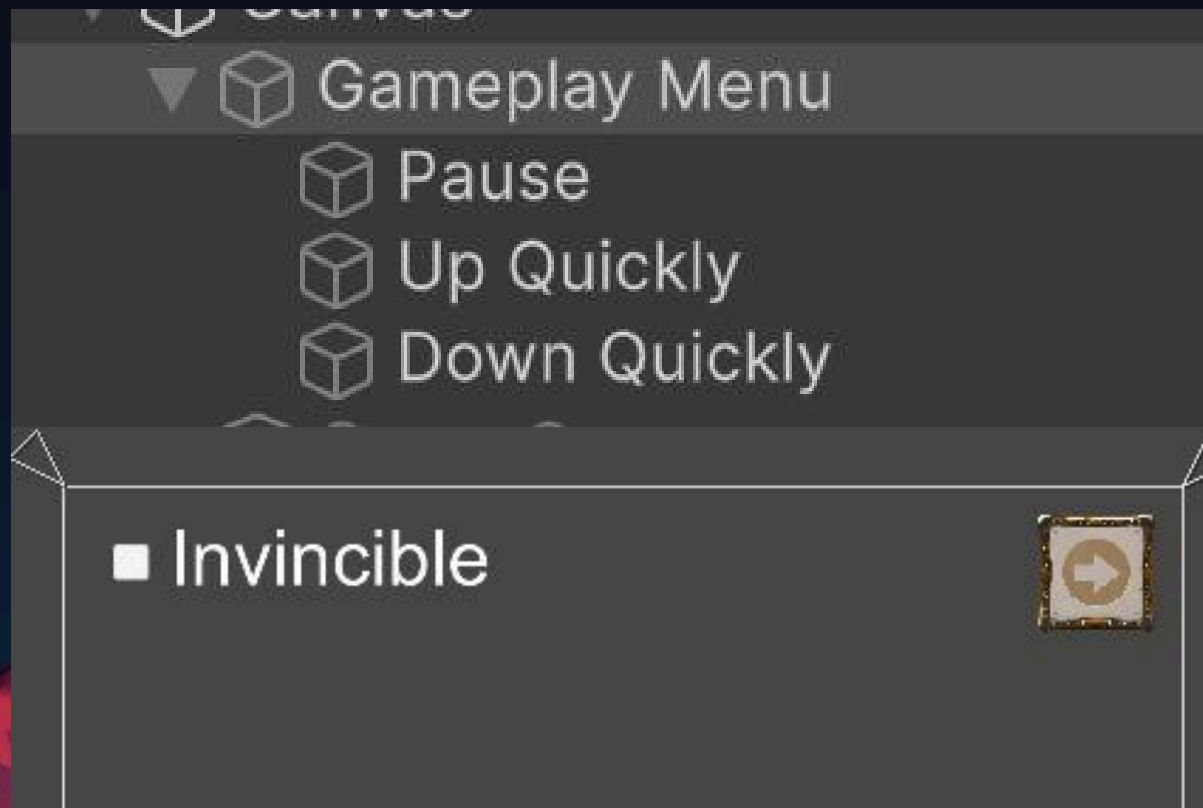


And modify level to this

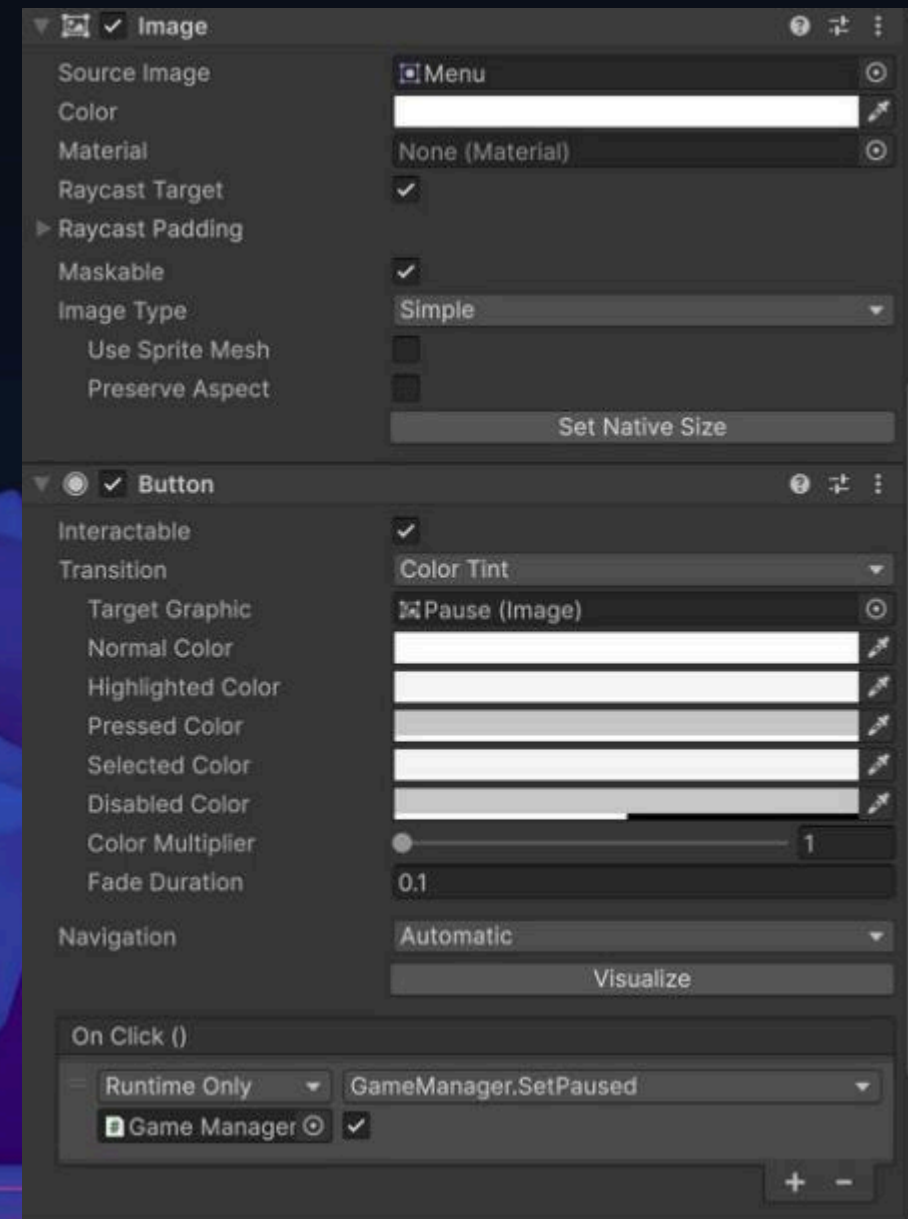


Game Mechanics

Add pause to the gameplay menu
and move it to the top left



Add this components to the
gameobject pause



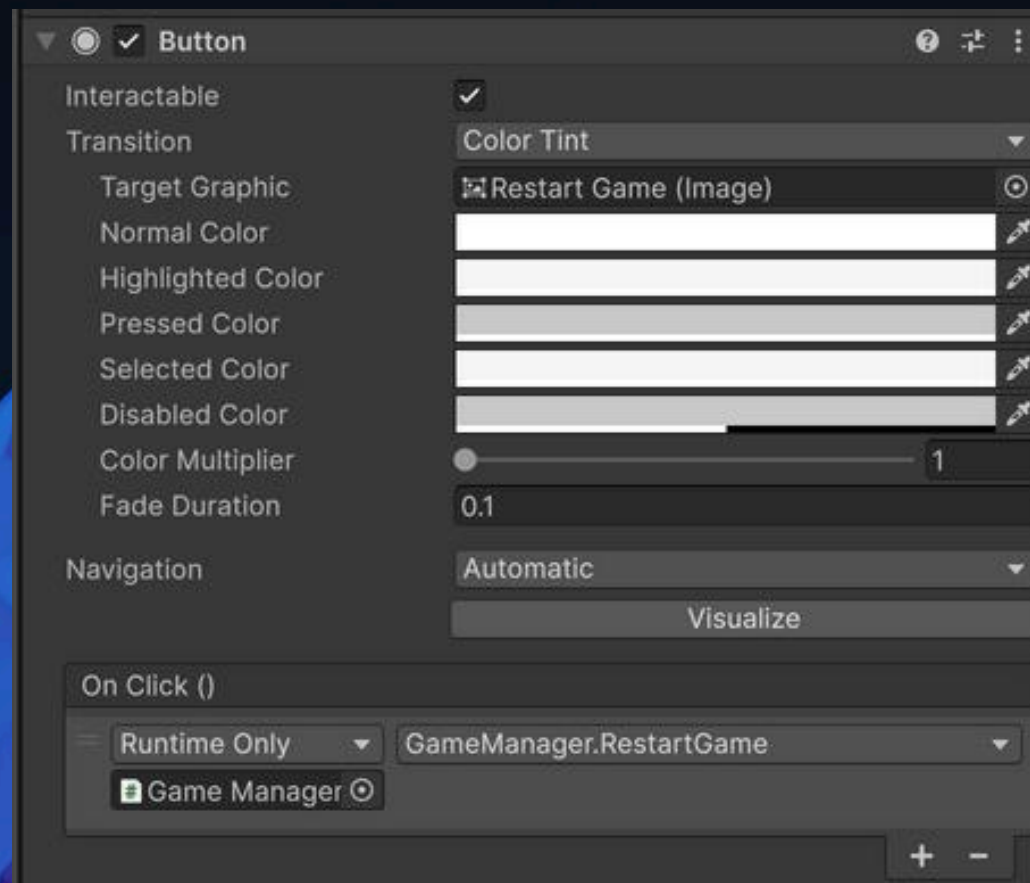
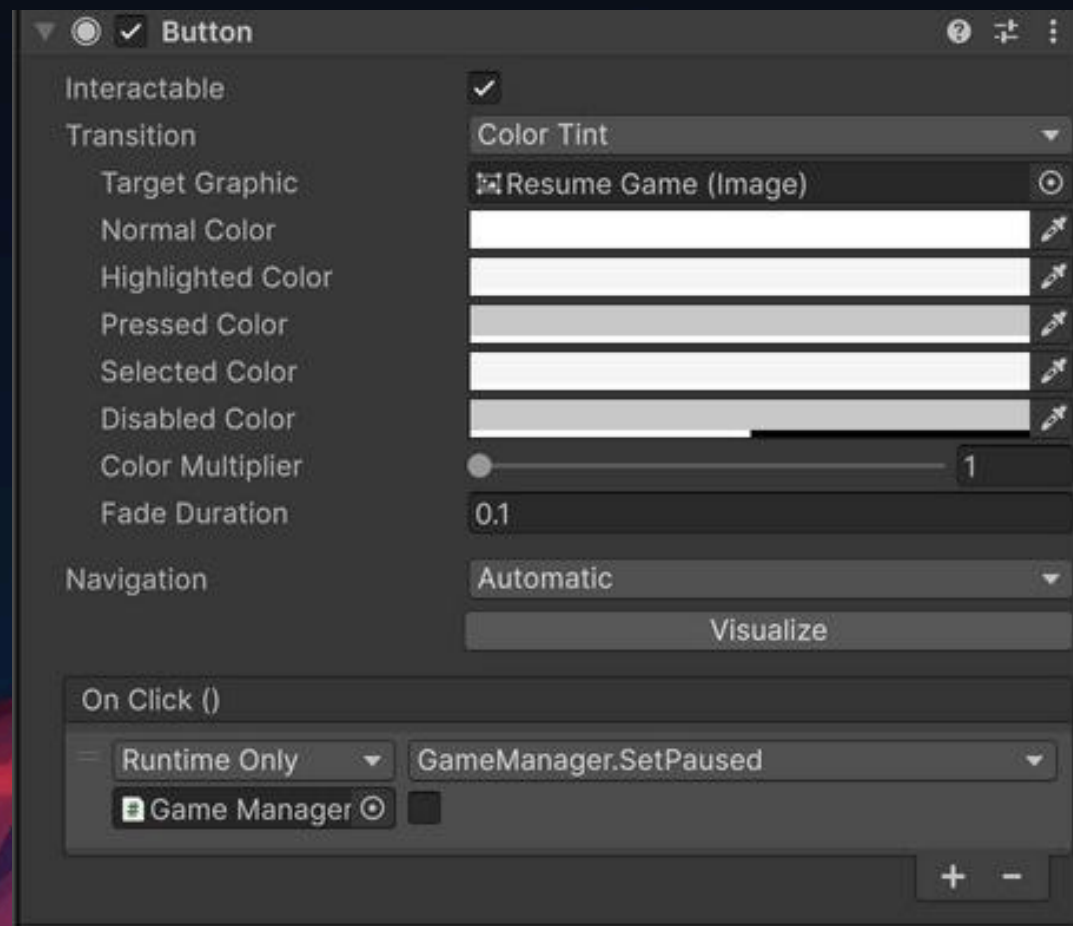
create 2 objects named
main menu and add this
gameobjects will look like
this



Creating End Screen

Add this 2 componnets

And set the game manager to
main menu and gameplay
menu

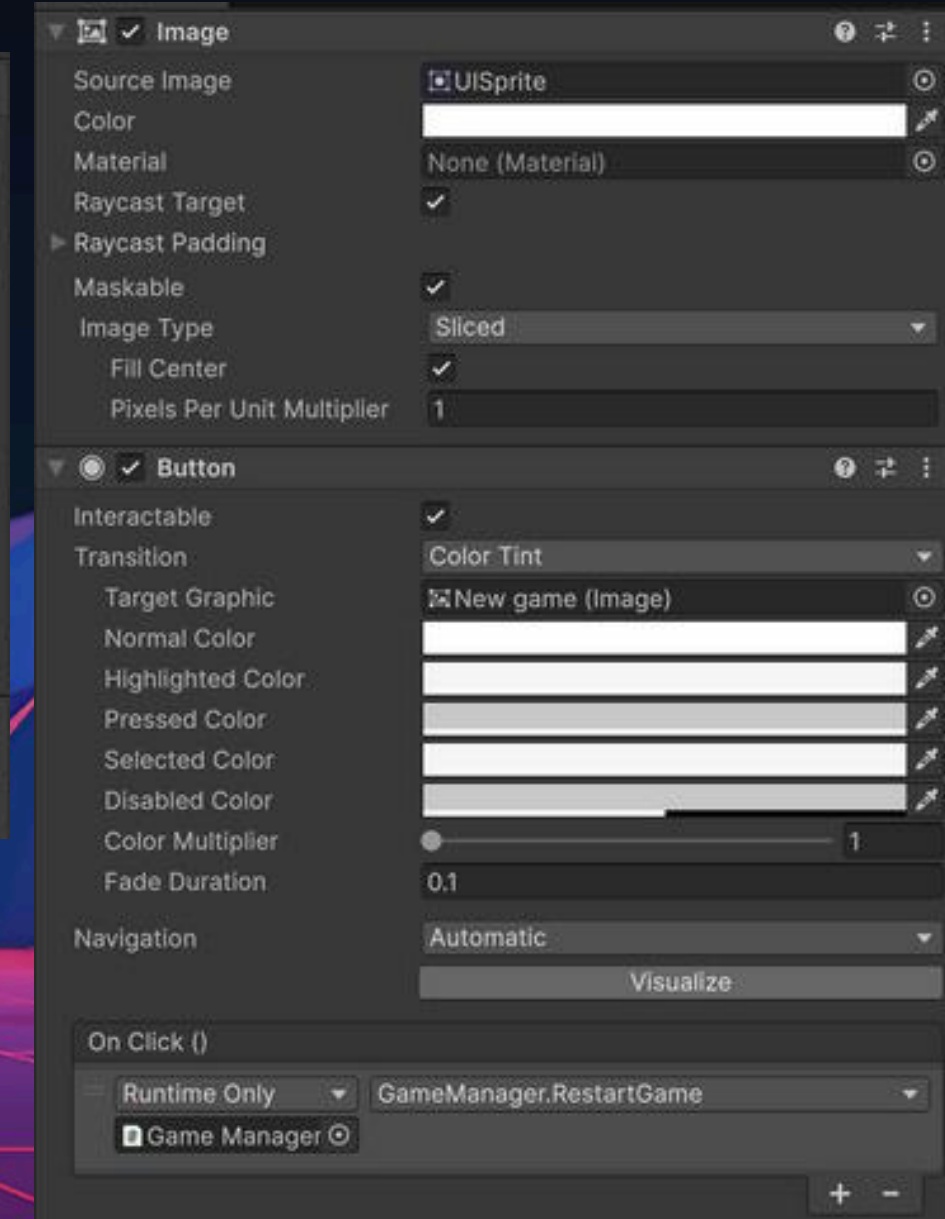
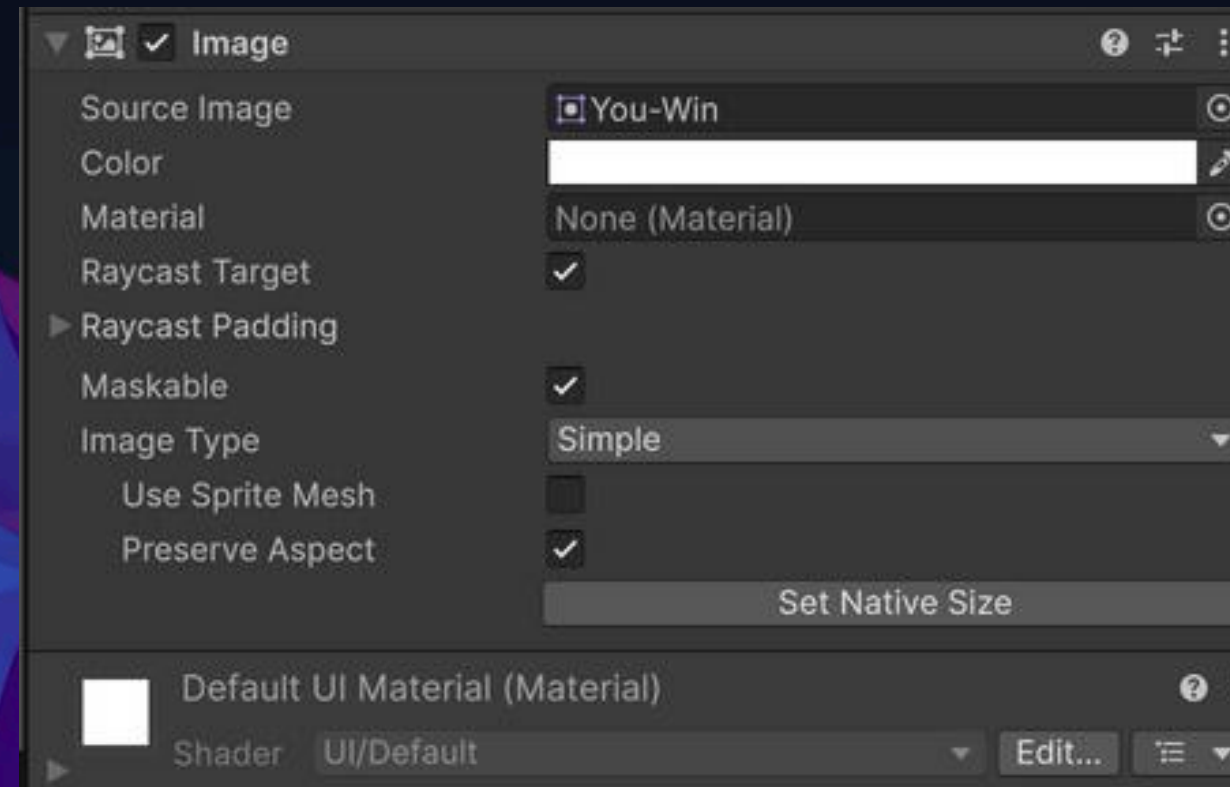
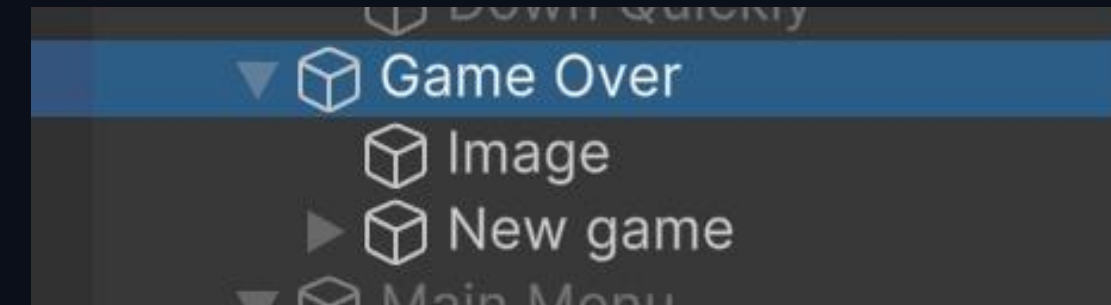


Creating End Screen

Create the game over screen

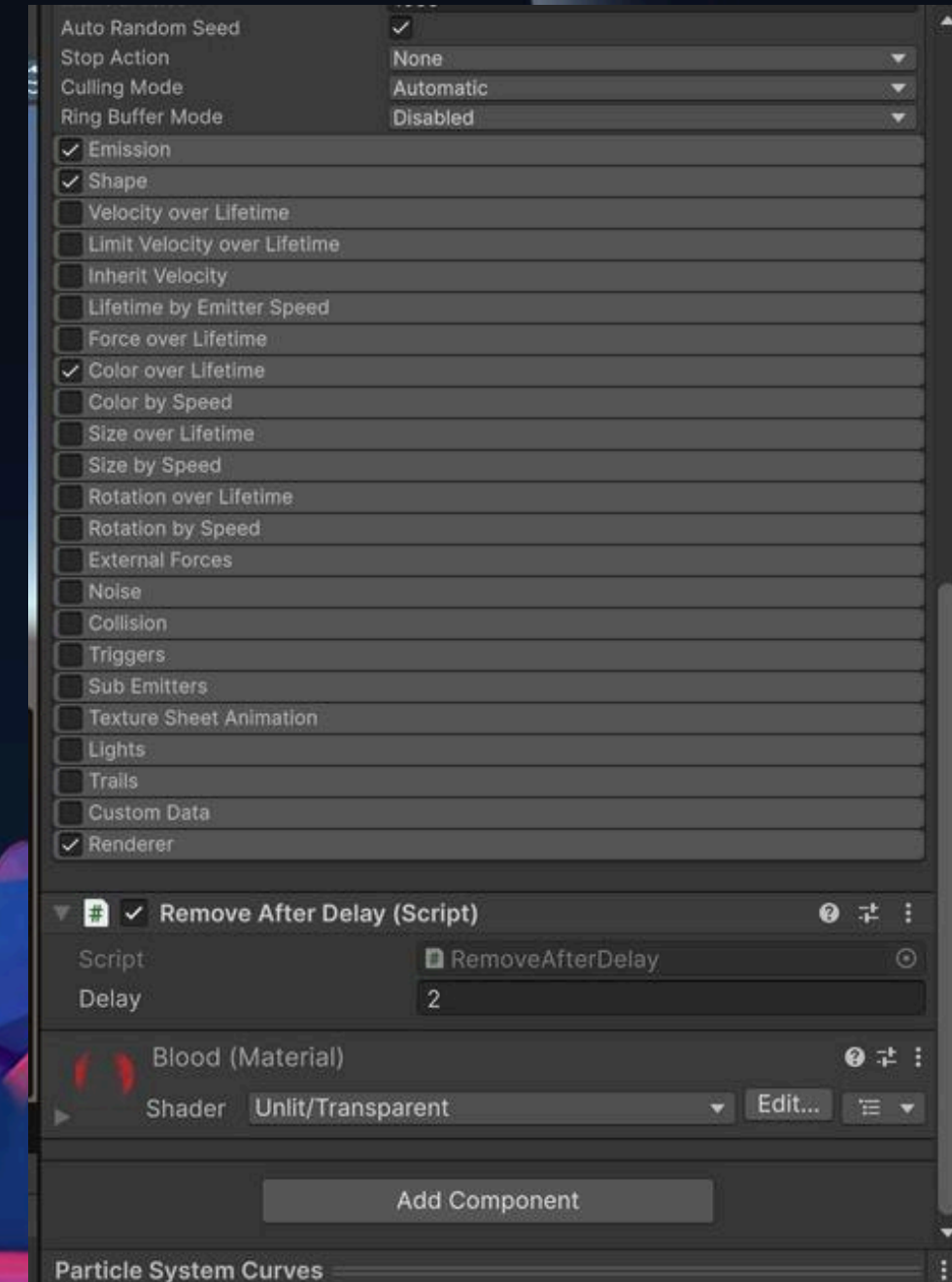
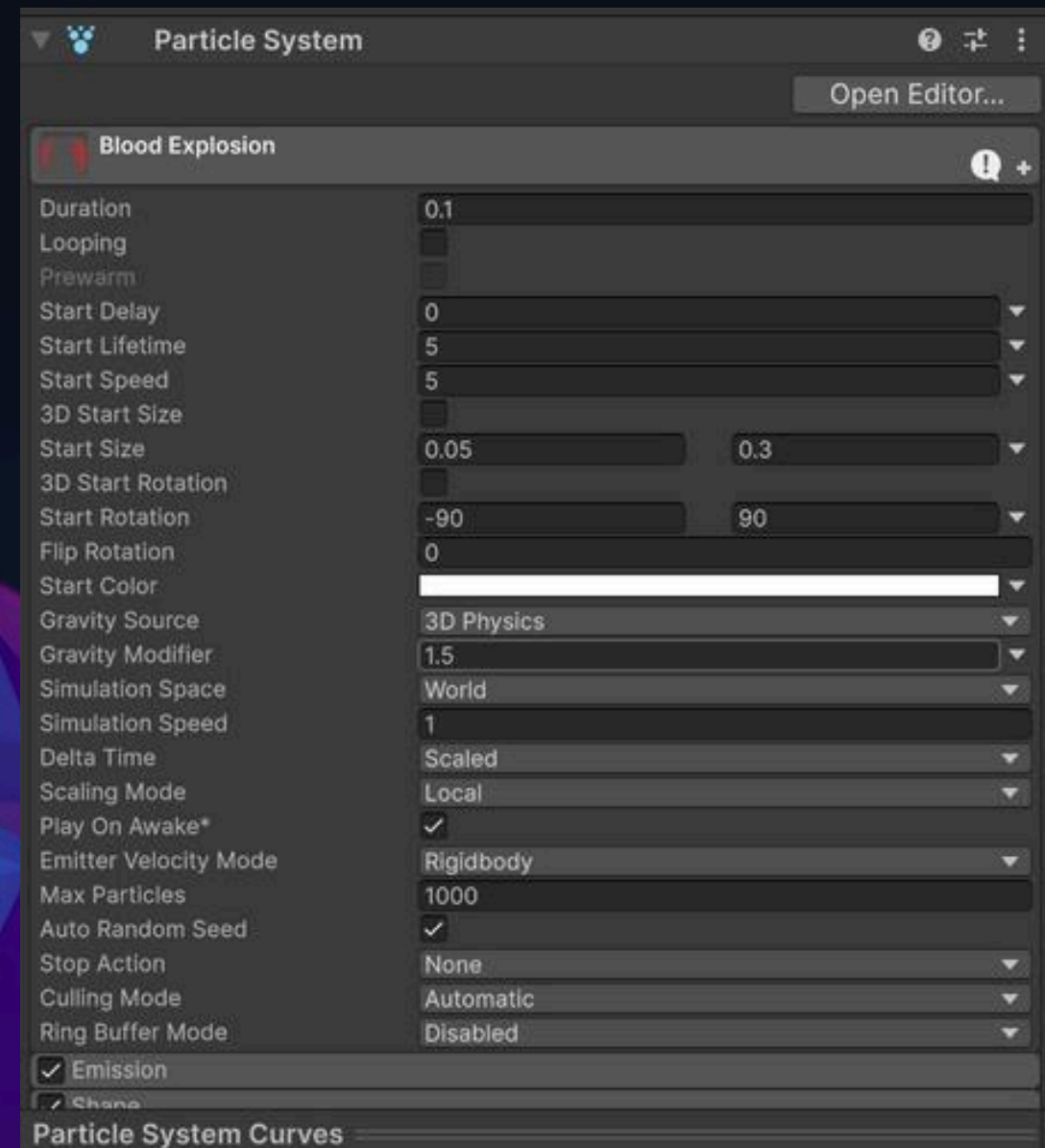
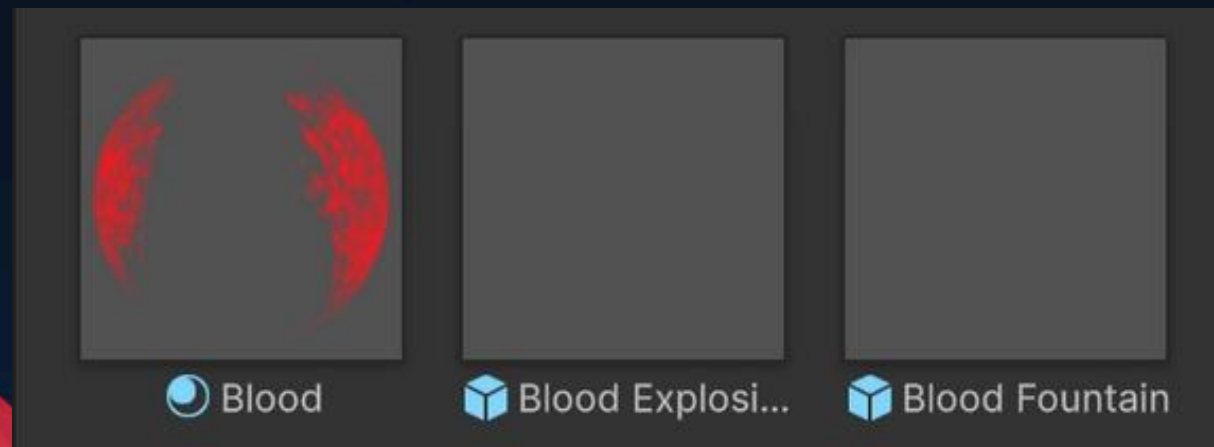


Create this 3 gameobjects



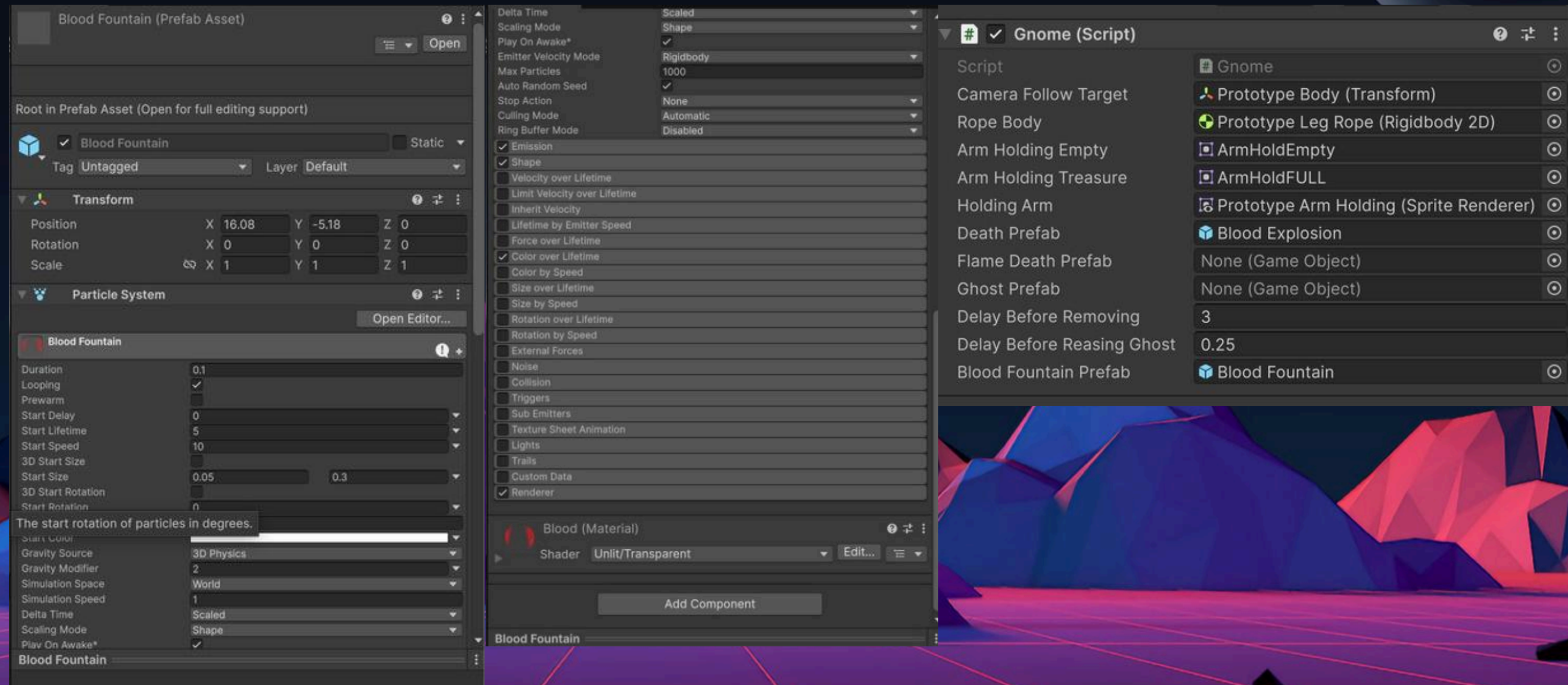
Setting Up Buttons

Create this prefabs called blood explosion and blood fountain into a particle system and copy this settings to blood explosion



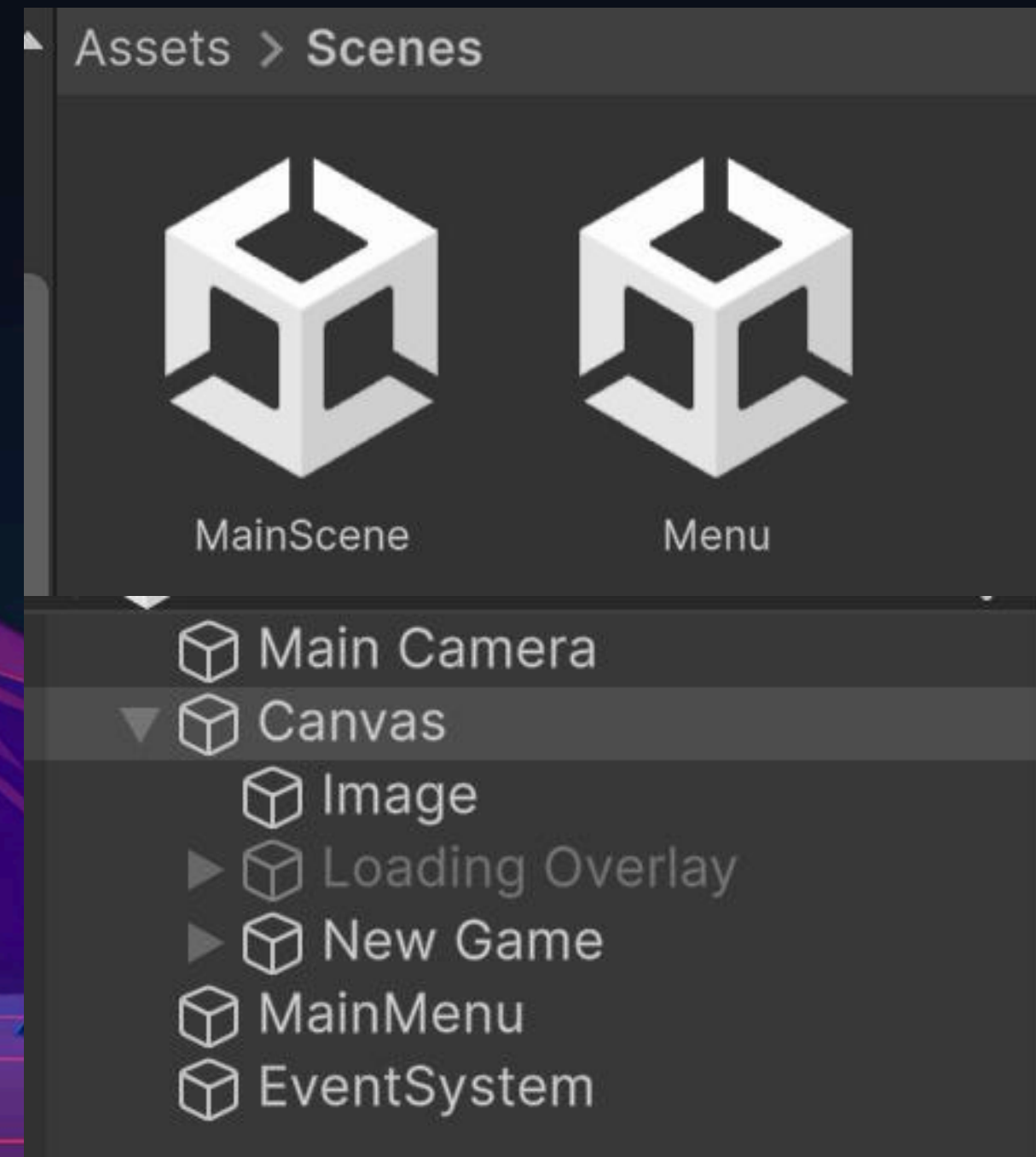
Setting Up Buttons

Copy this settings to blood fountain
and update the gnome script in the
Gnome prefab



Game Main Menu

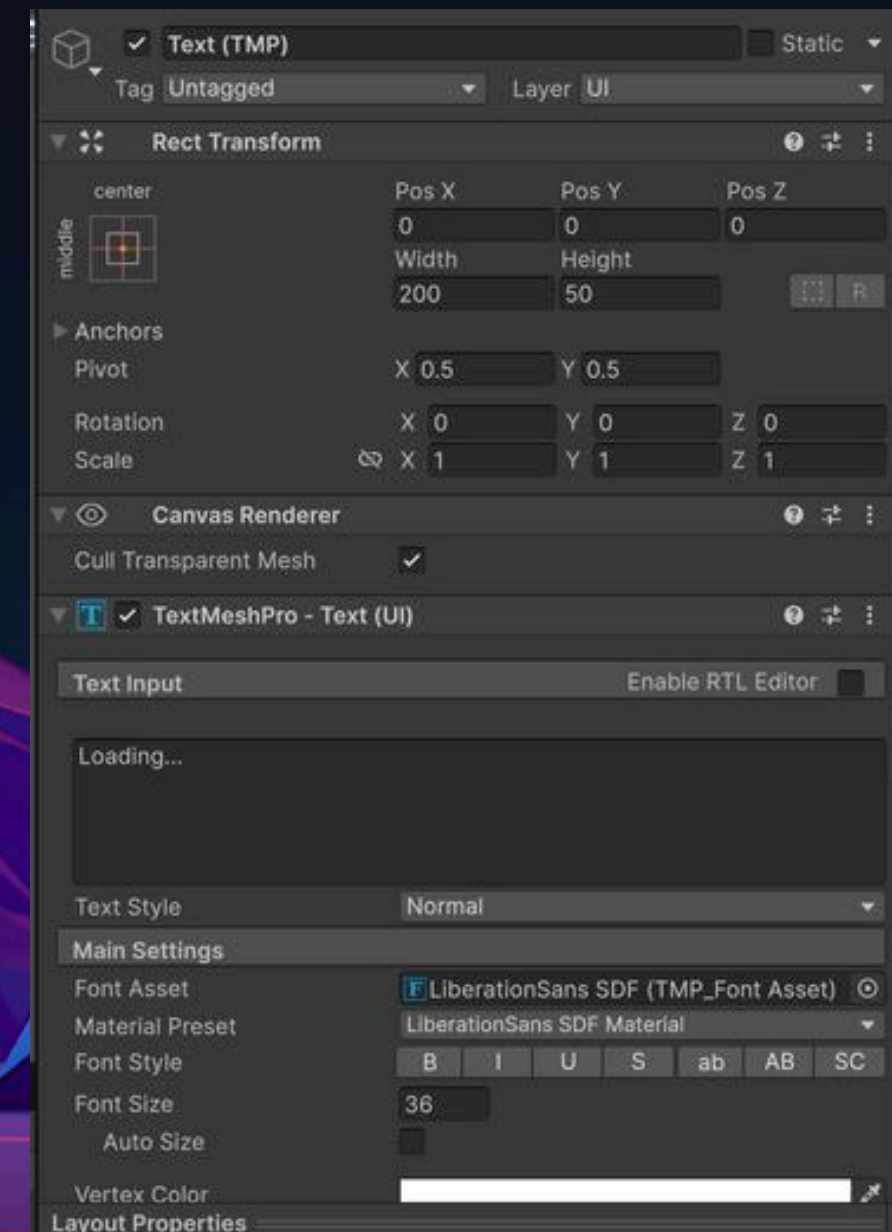
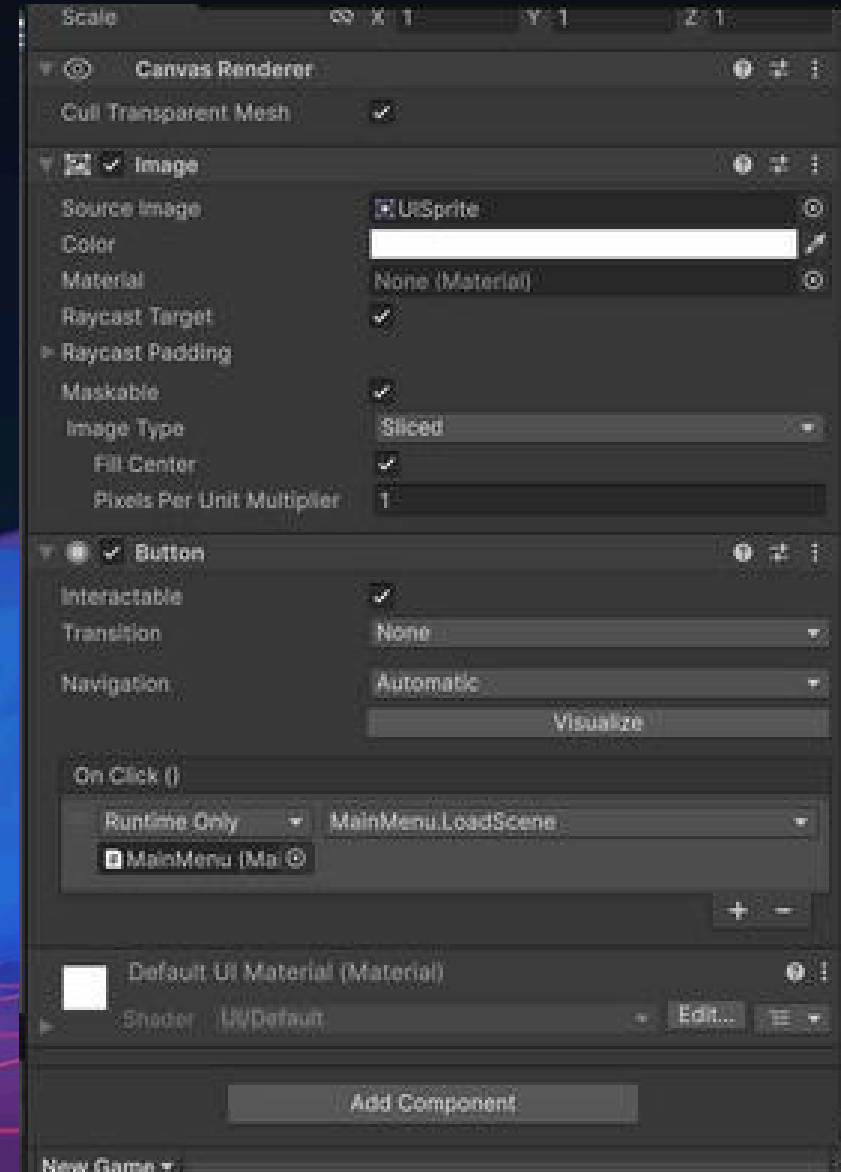
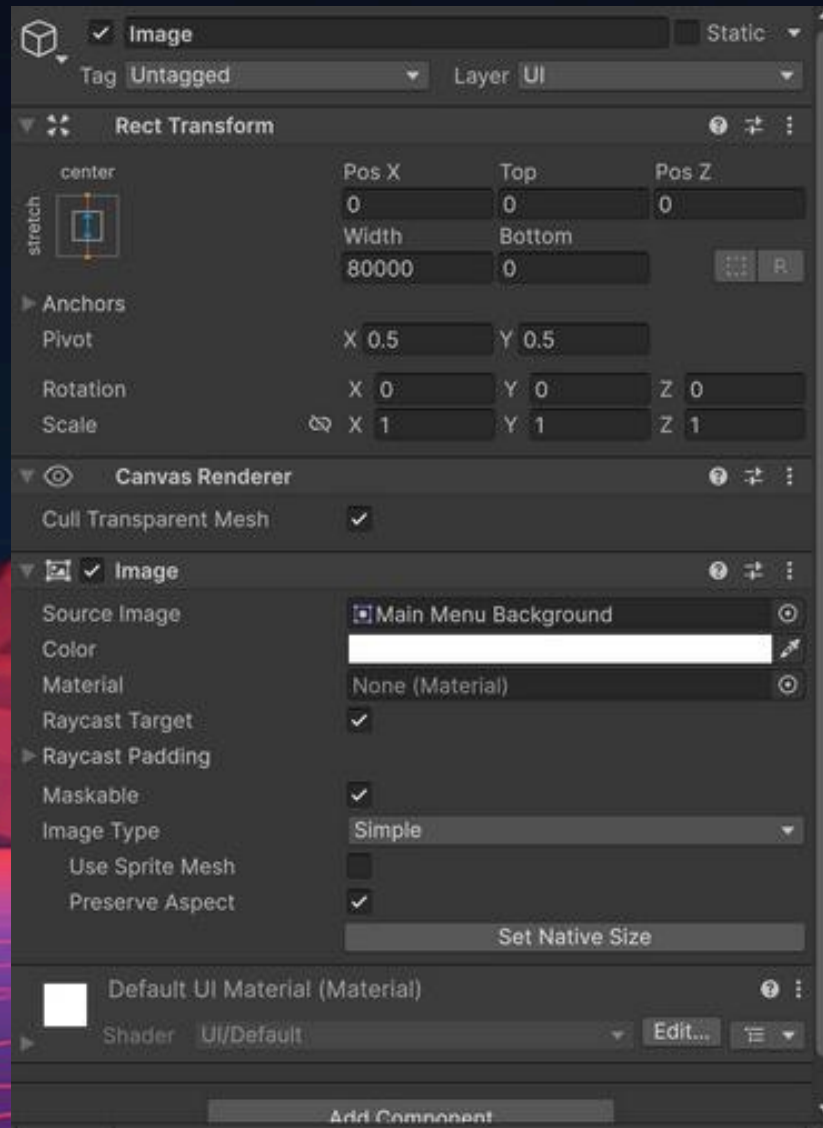
Create new scene named
menu and make this
objects



Game Main Menu

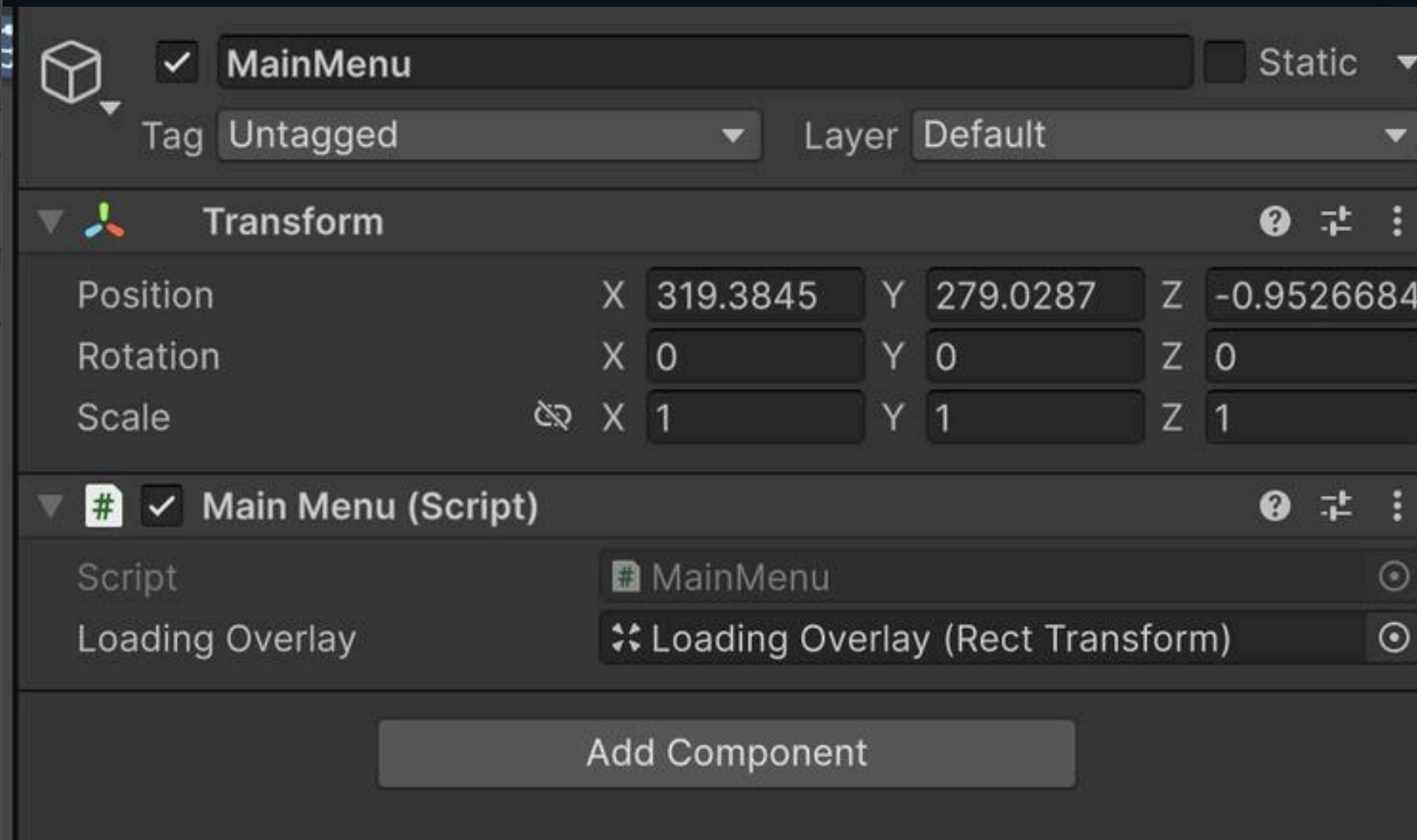
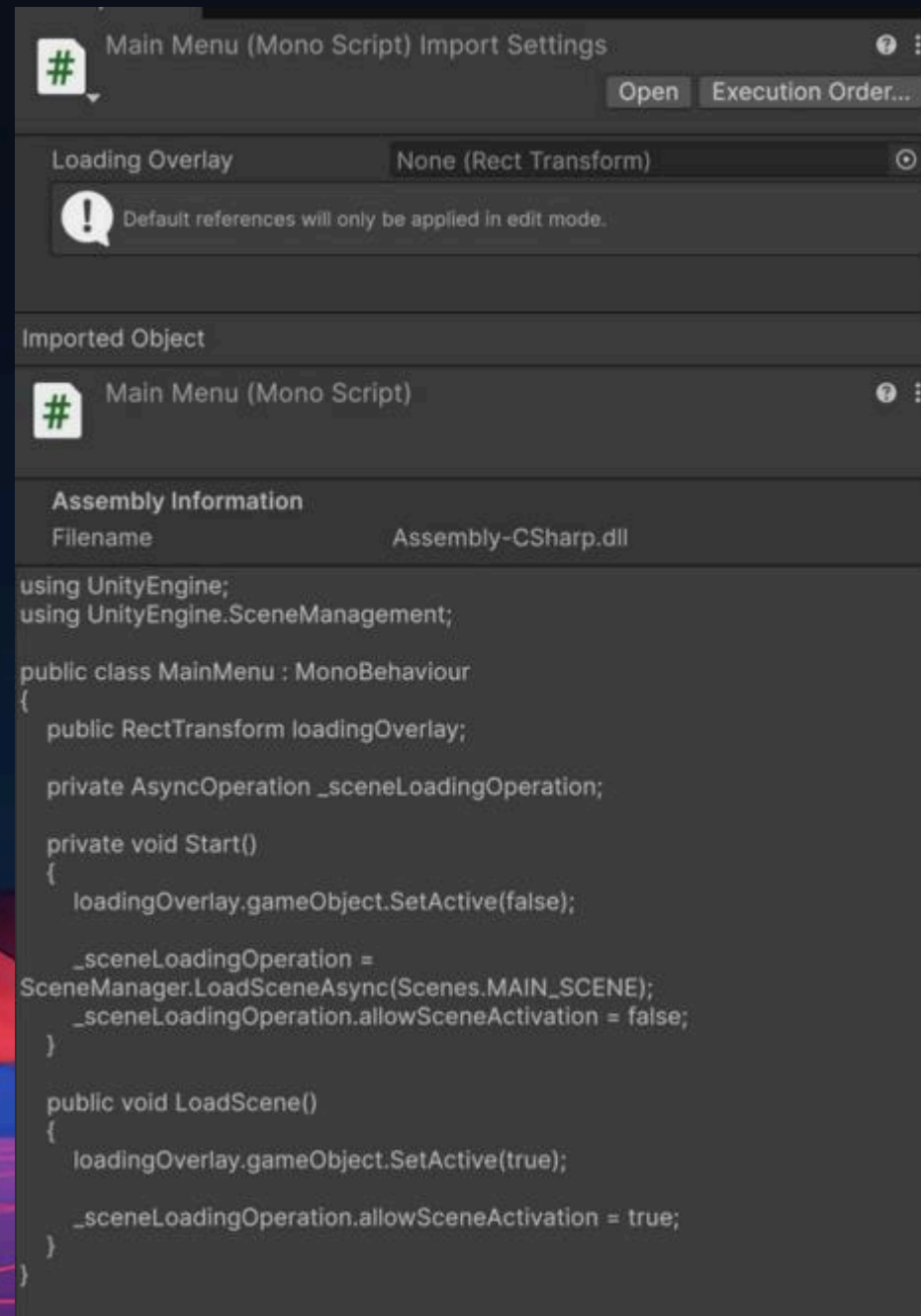
Make this image settings and in the pause game add a new game button and add this buttons

make this in the loading overlay text will look like this



Game Main Menu

copy paste main menu code and
move it on the main menu object



Final Output

There you go, now
you've created a
game

